



Doubts with Lakshay Bhaiya [Week 7]

Special class



Pointers - Class 1 [Join Here]

Special class

Prime number

→ ① Naive

2 → N-1

↳ $\cdot$ $\rightarrow$

②

Sqrt

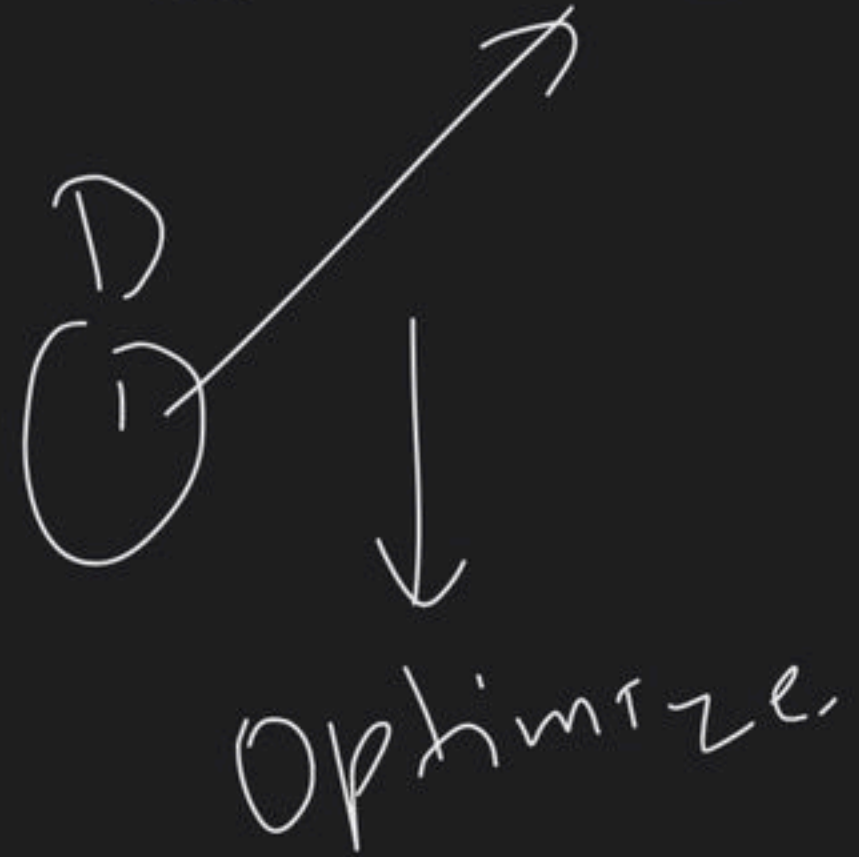
2 → $\leq \sqrt{N}$

↳ $\cdot$ $\rightarrow$
 $\cdot$

③

Sieve of Eratosthenes

☆☆



2hr

N

1 → N

↓

Are primes

$\Rightarrow$ ①

inner loop ✓

②

Outer loop ✓

$$\Rightarrow \text{LCD}$$

$$\stackrel{=}{\hookrightarrow} \text{LCD}(a, b)$$

$$\Rightarrow \text{gcd}(a-b, b) \quad a > b$$

$$\text{gcd}(b-a, a) \quad b > a$$

$$\Rightarrow \frac{0}{0} \neq$$

$$(a \% M * b \% M) \% M = (a * b) \% M$$

$(\text{within } M) * (\text{within } M)$
 $\downarrow \quad \downarrow$

out of Bound ~~△△~~

M
 $\downarrow$

Maths ~~OP~~

+
*

Code
Sham
Sen
★

$$\text{int } a = 2^{11}$$

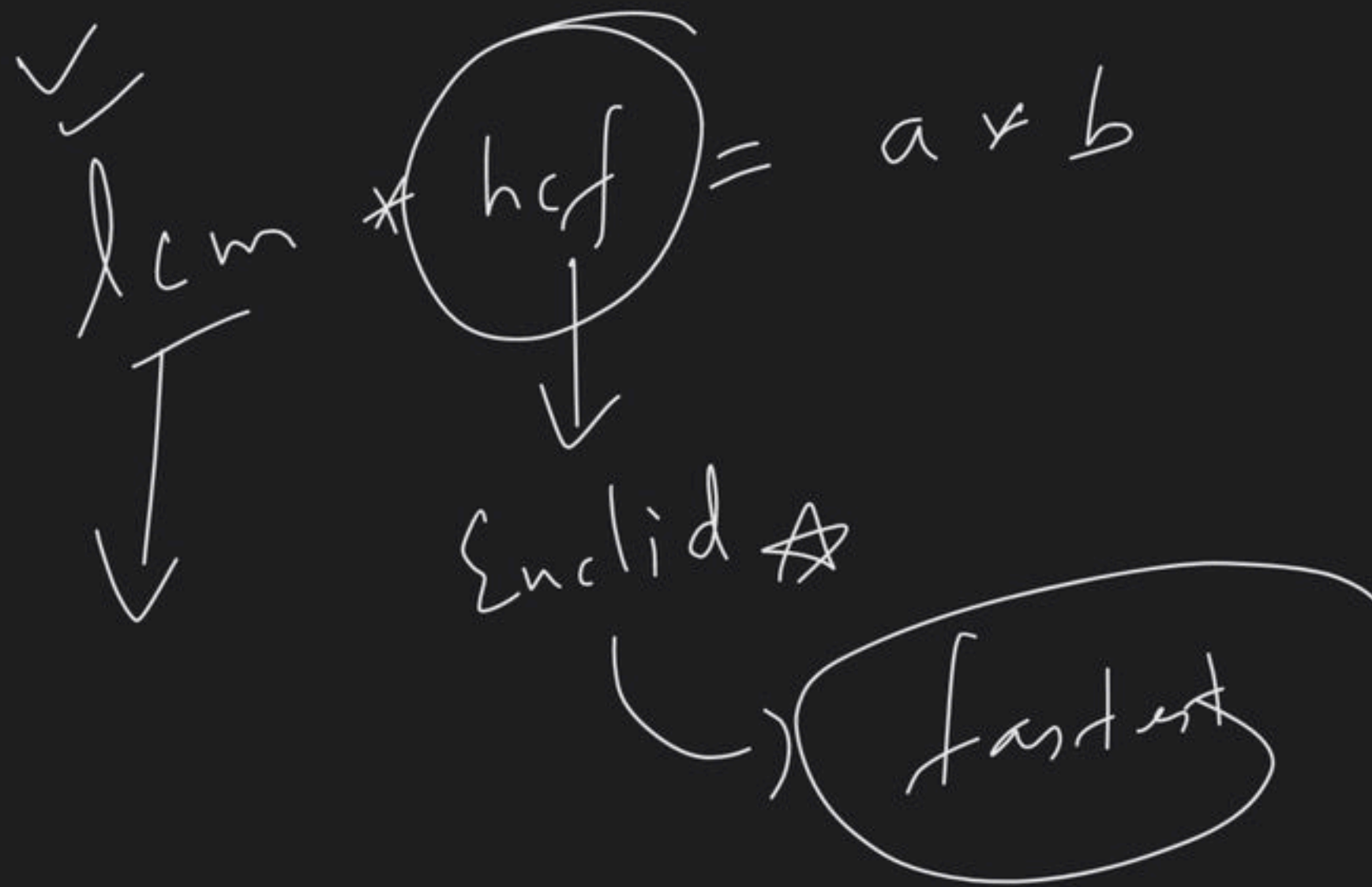
$$\text{int } b = 2^{13}$$

$$\text{int } c = a * b$$

$$M = 10^9$$

within
=

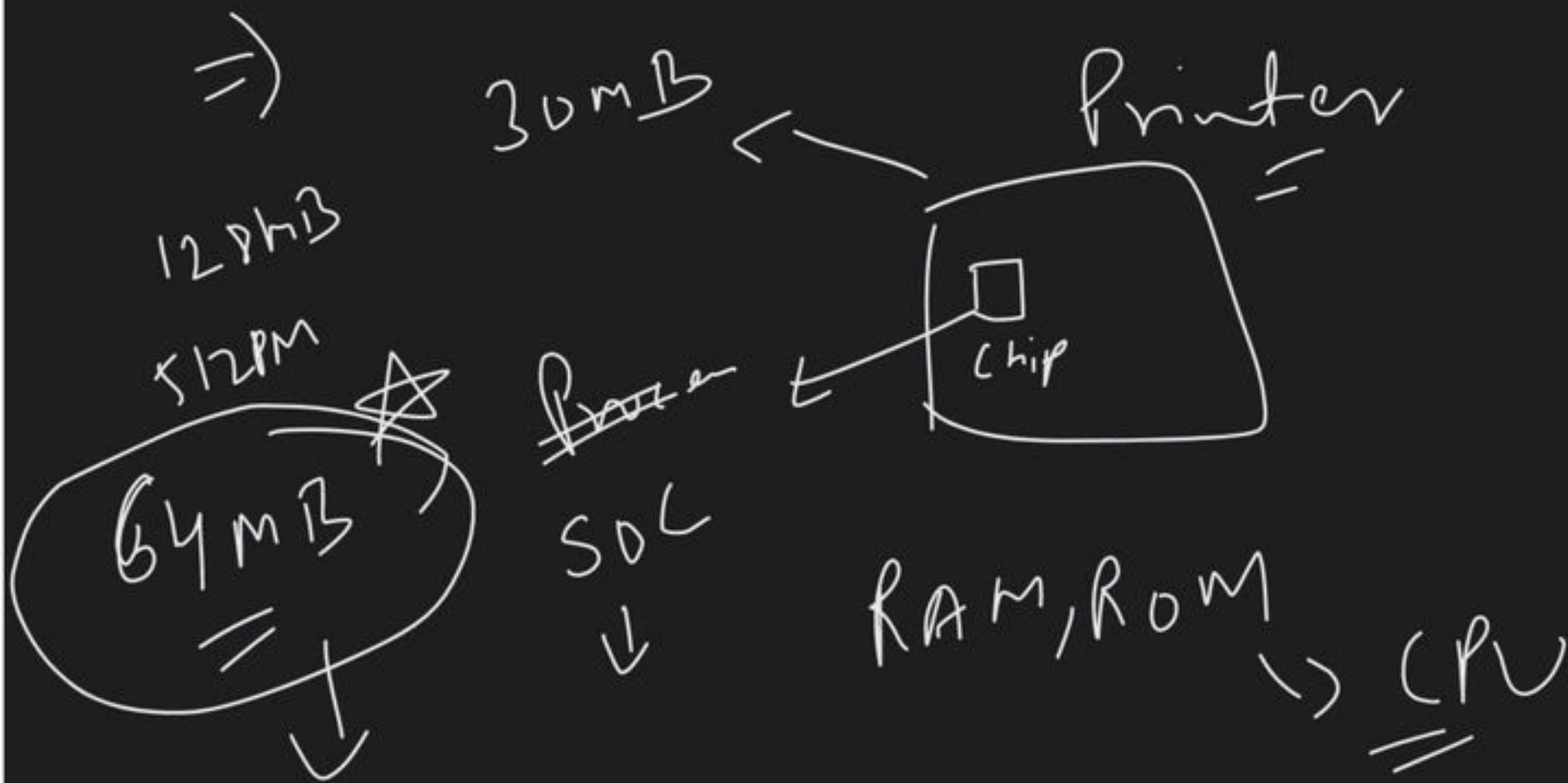
$$\Rightarrow c = \left((a \% M) * (b \% M) \right) \% M$$



$$= a^b$$

System — software

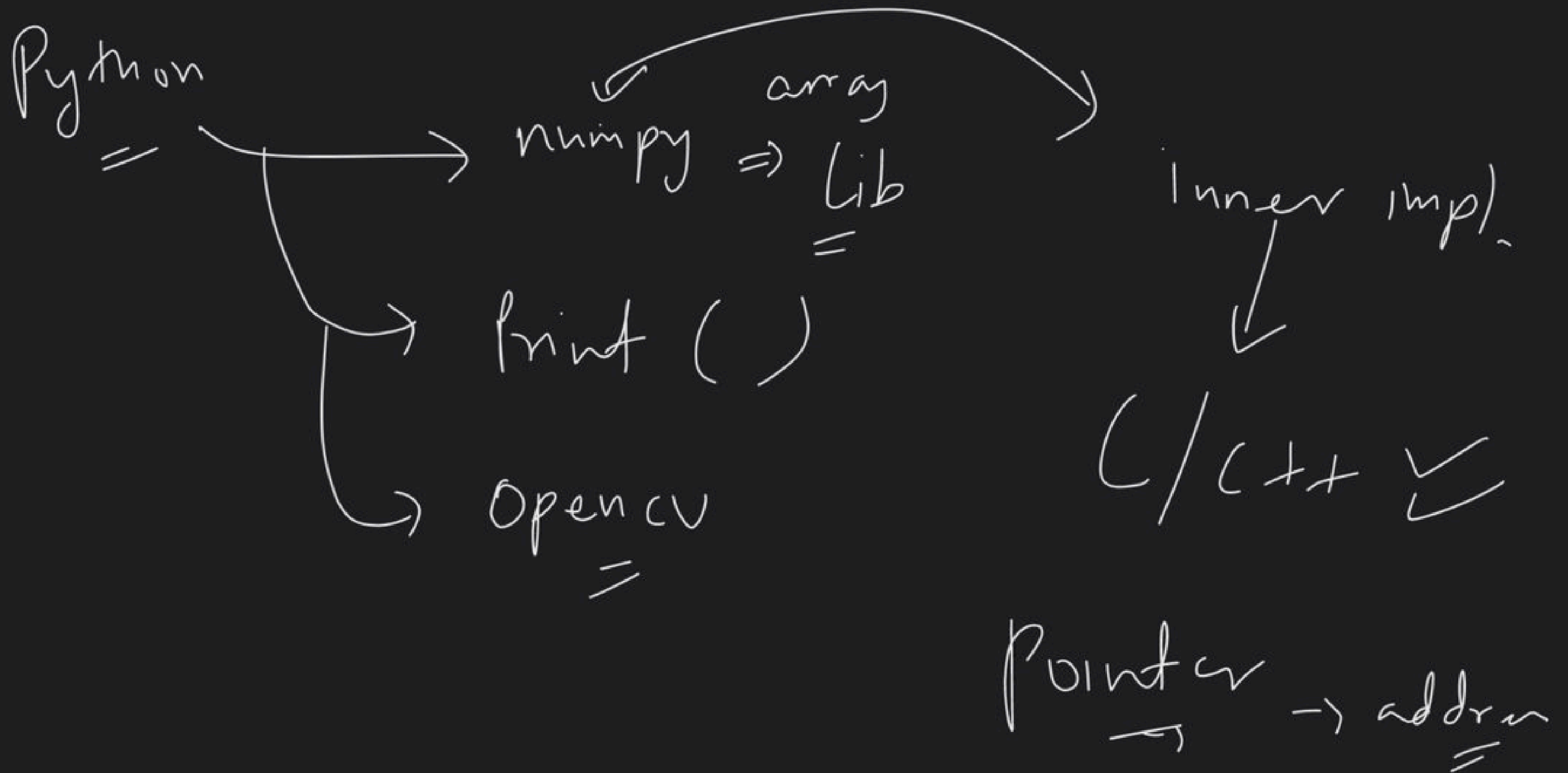
C++



Memory
↓

Inner Level =

Python
Java

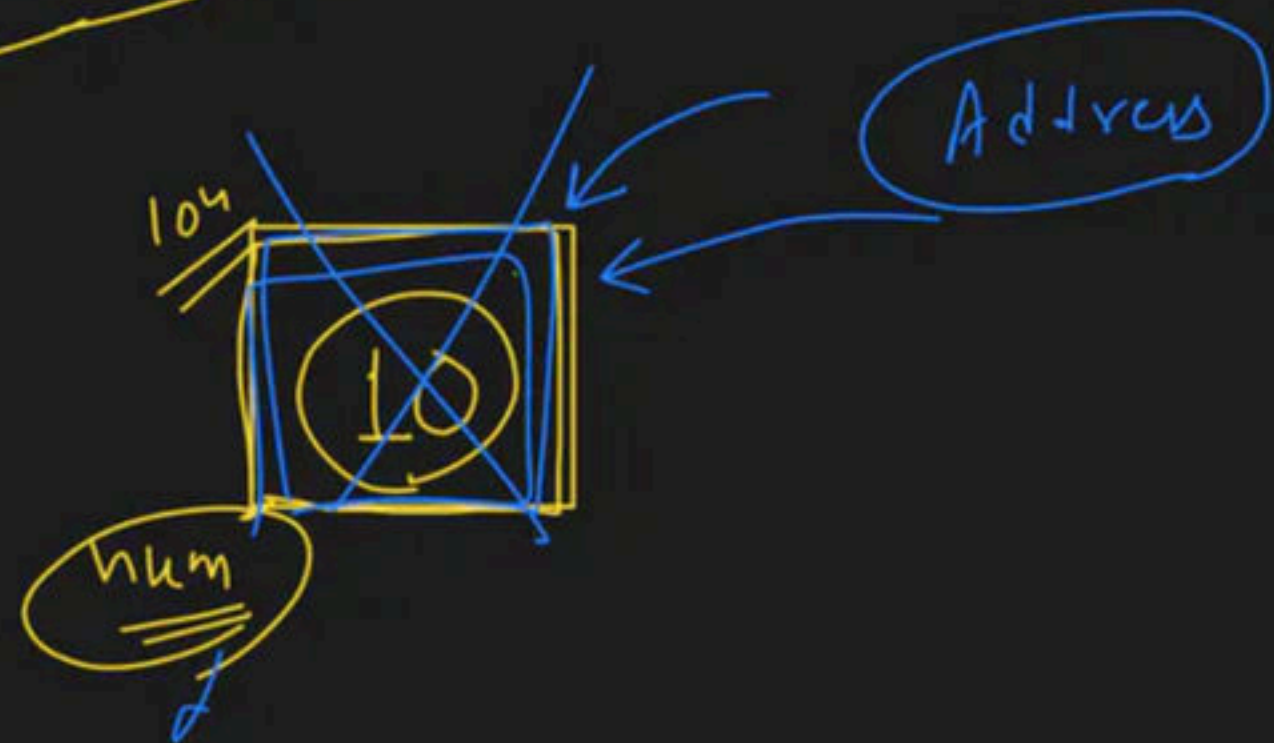


→ Pointers:-

variable $\longleftrightarrow$ store address of
other variable

⇒

int num = 10;



print address

int num = 5

address of operator

24

104
5
1 byte

Symbol table

num → 104

&num

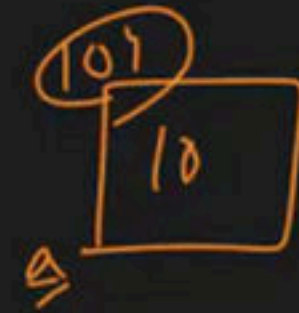
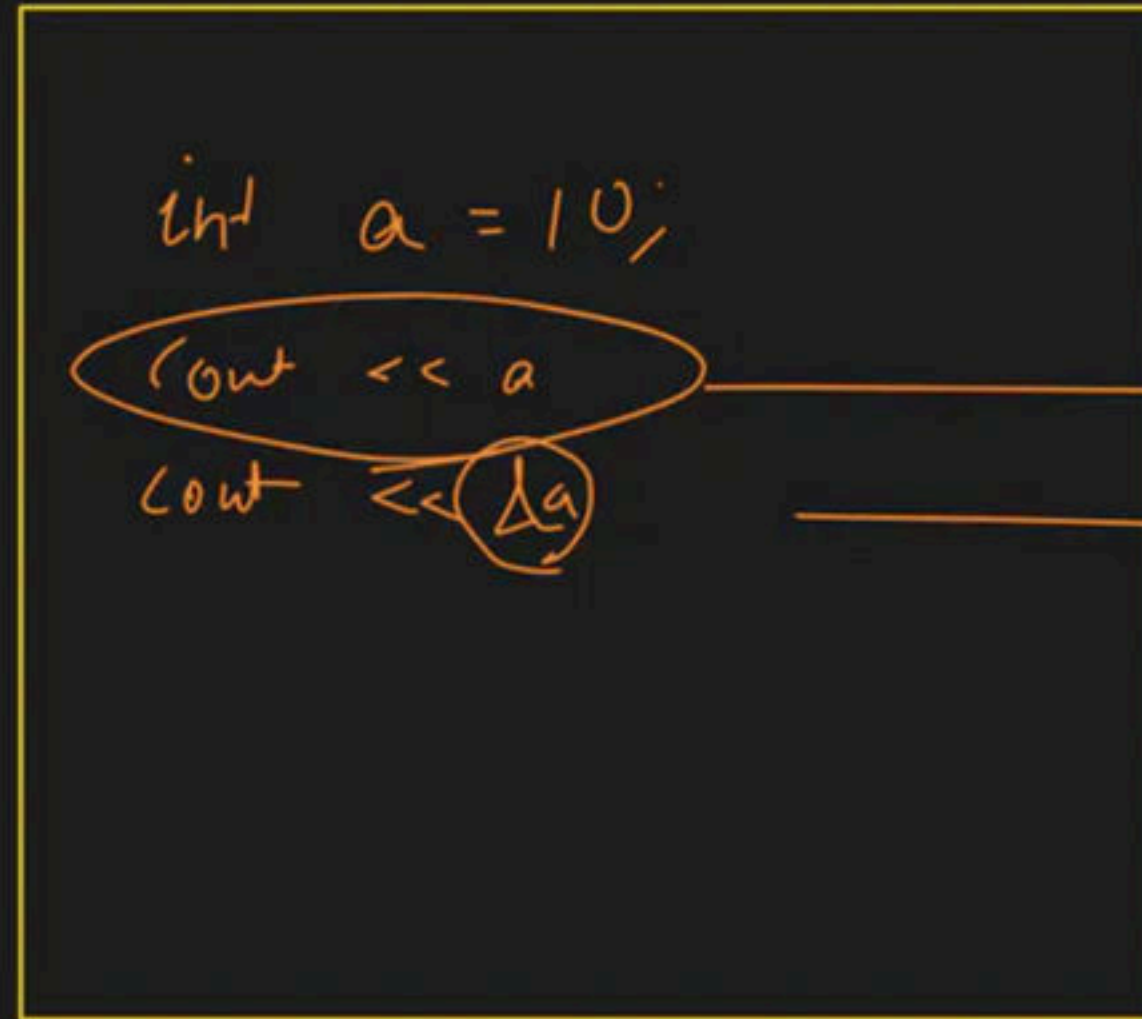
104 → hexadecimal

int a = 5;

216
a
= 5

S.T

a → 216



! → 10

! → 101 =

Pointer

bool *

Creation

pointer name

int *

ptr

== ;

address

variable name

pointer to integer data

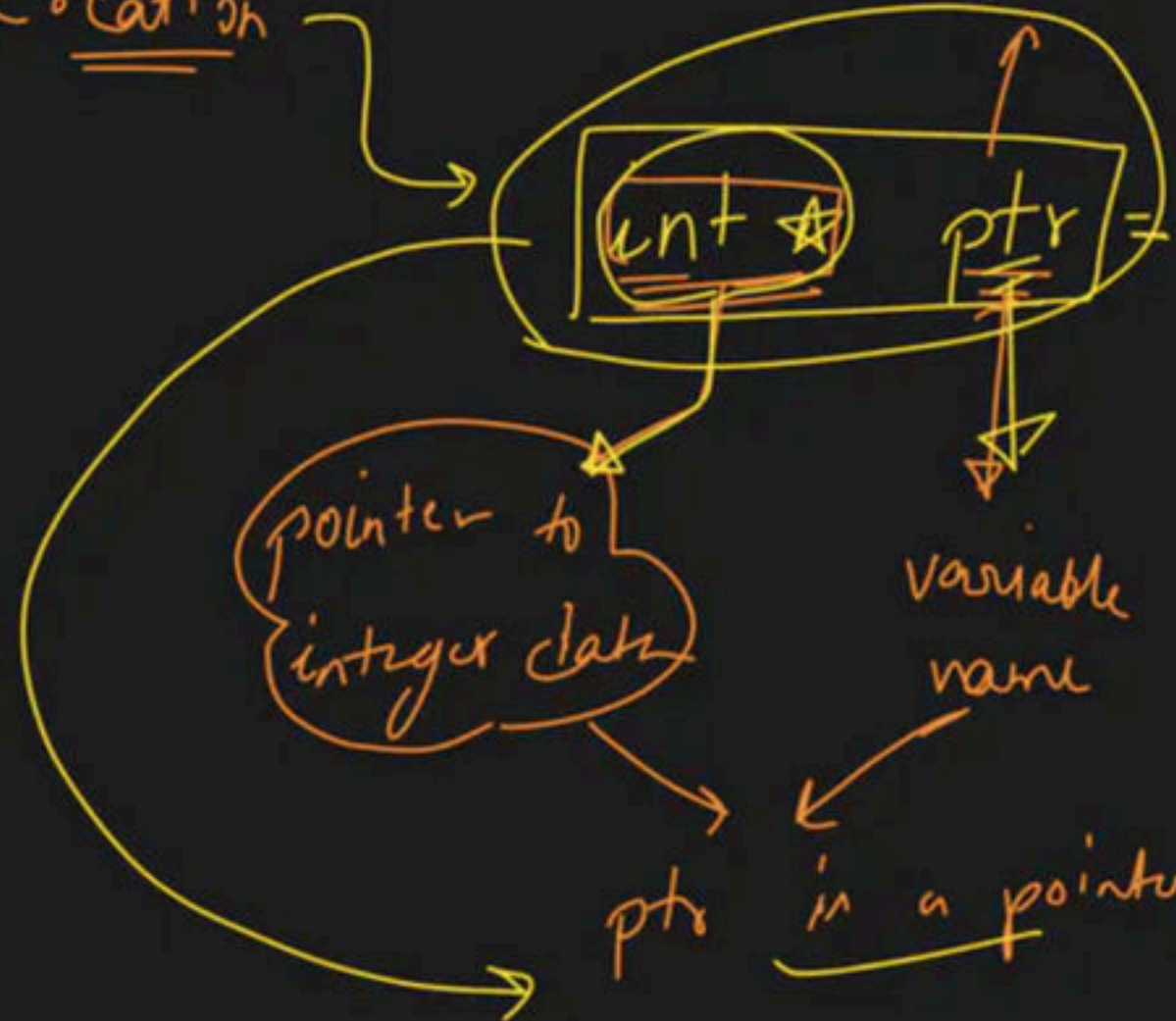
ptr in a pointer to integer data

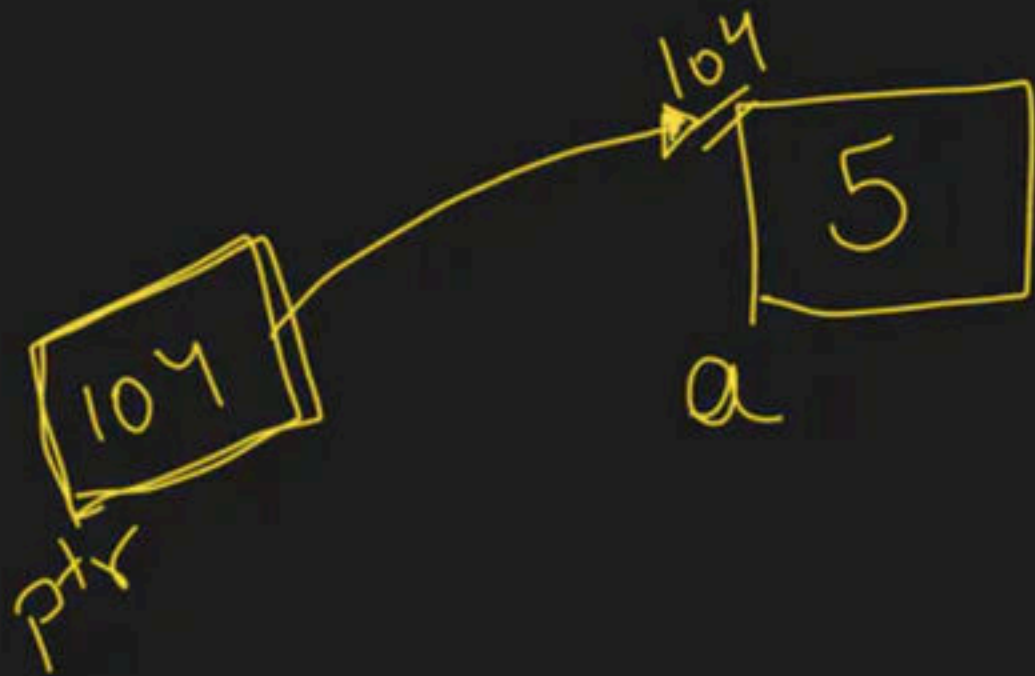
char *

pointer to char data

long *

short *





```
int a = 5;
```

```
int * ptr = &a;
```

Diagram showing the declaration of a pointer variable. The variable 'ptr' is of type 'int' (indicated by the underline) and is assigned the address of variable 'a' (indicated by the ampersand and 'a'). Arrows point from the underlined 'int' to 'ptr' and from '&a' to 'a'.

int a = 10;

Why?

Crash

int * ptr = &a;

Acc

→ value stored at address stored in ptr
↳ dereference → *

cont * ptr
↓
10

2016

ptr

1008

1008

10

a

int a = 5

~~int~~

int * ptr = &a;

int * ptr

gib. int &ptr

Why

Dynamic Memory

2016

ptr

1008

1008

a

5

ptr → 1008

&ptr → 2016

*ptr → (I) address ptr has (II) value ptr

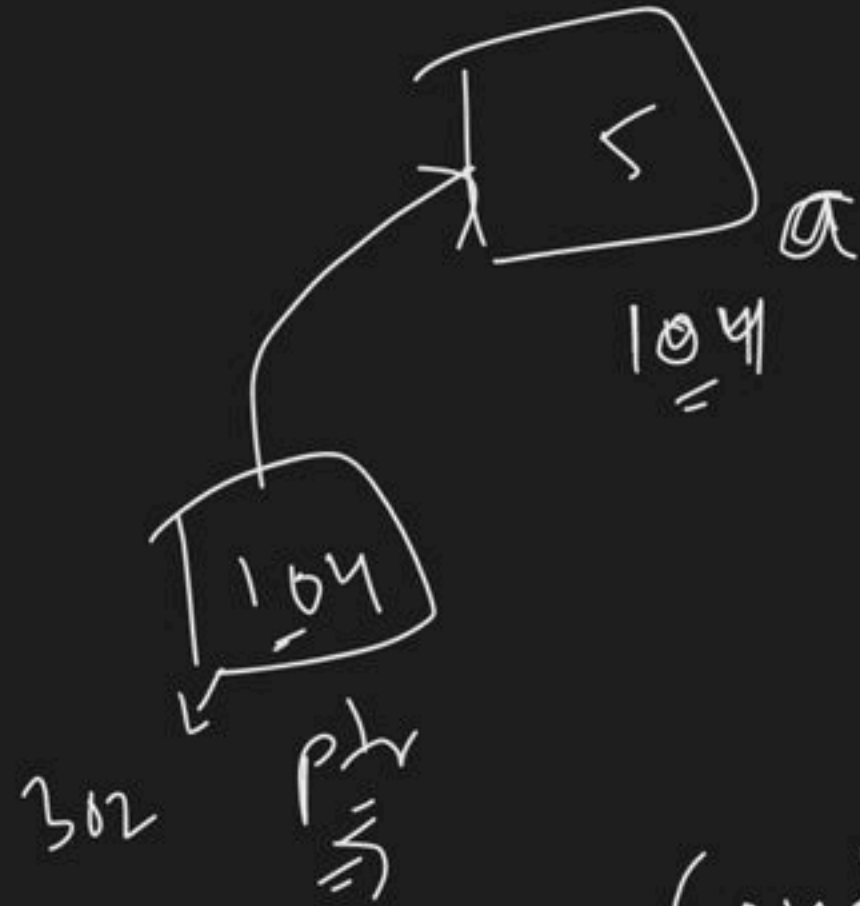
→ 5

a → 5

&a → 1008


```
int a = 5;
int *ptr = &a;
```

↳



cout << ptr; → 104
 &ptr ⇒ 302

~~int~~

int *ptr = &a

int *ptr = a

int ptr = &a

(*)ptr

32 or 64 bit



```
int a = 5;  
int * ptr = &a;
```

sizeof(ptr) => ~~4~~
8

```
char ch = 'K';  
char * cptr = &ch;
```

sizeof(cptr) => ~~1~~
8

```
long lachi = 10;  
long * lptr = &lachi;
```

sizeof(lptr) => 8

h/w

32 vs 64 bit $\rightarrow$ need $\rightarrow$

$\Rightarrow$ 32 bit

$\downarrow$

pointer

$\hookrightarrow$ 4 bytes
=
=

32 bit

$\rightarrow$

1	0	1	1	1	1
---	---	---	---	---	---

 ϕ 31

why

dynamic memory allocation

2^{32} address
=
=

8 byte
=
=

illegal memory access

int

ptr

declare

int a;

BAD Practice

Runtime Error

S.F

null ptr

int *ptr = 0;

nullptr

~~int *ptr;~~
~~cout << *ptr;~~

garbage

int *ptr;

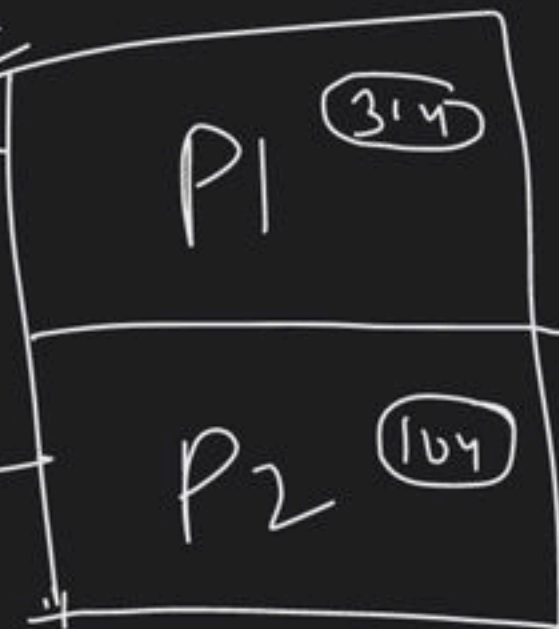
wild pointer

uninitialized ptr

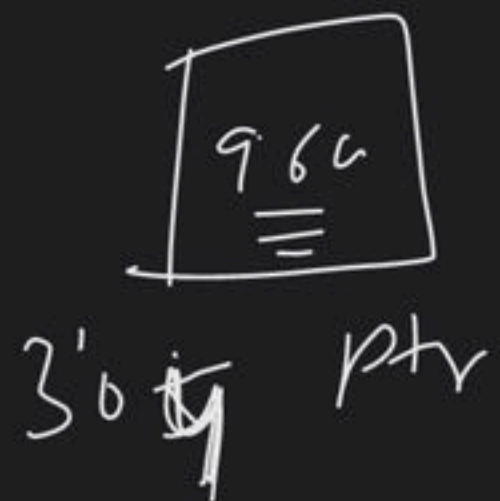
int *ptr = 0;

MS worth

Chrome



RAM



int a = 0;
if (ptr != 0) ✓
a = *ptr; ✓

int *ptr = Null; ✓
=

int *ptr = 0; ✓

int *ptr = nullptr; ✓

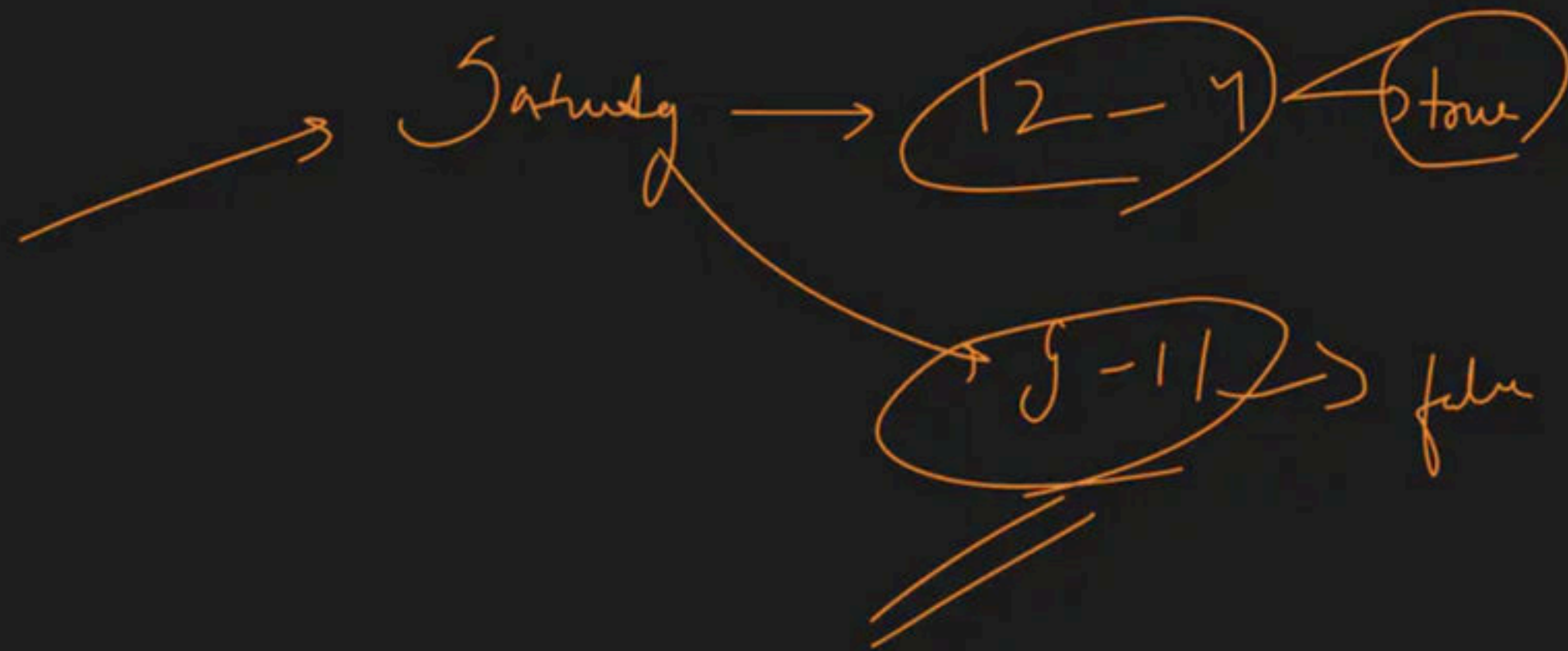
⇒

if (ptr != Null) ✓

ptr



Expected
Union



①

`int a = 100; //`

`int *ptr = &a //`

`a = a + 1;`

`ptr = ptr + 1;`

`a` → ?

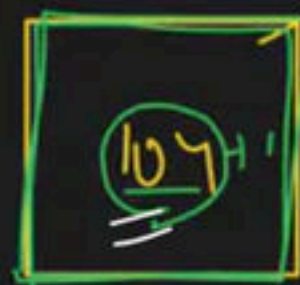
`ptr` → ? 108

104 + 1
= 105 x

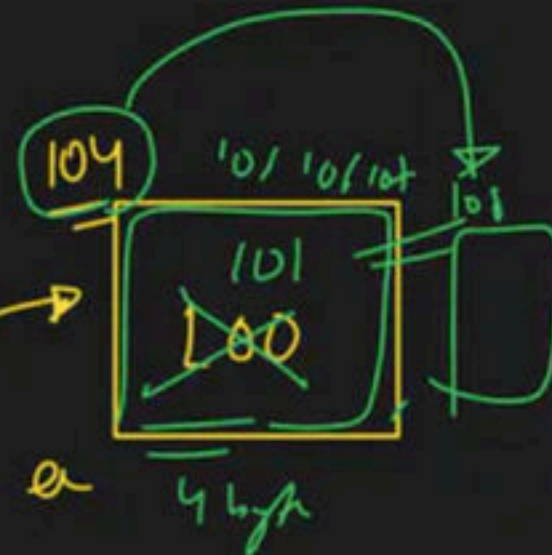
1 byte

bool → 1 bit
1 byte → 8 bits

1008

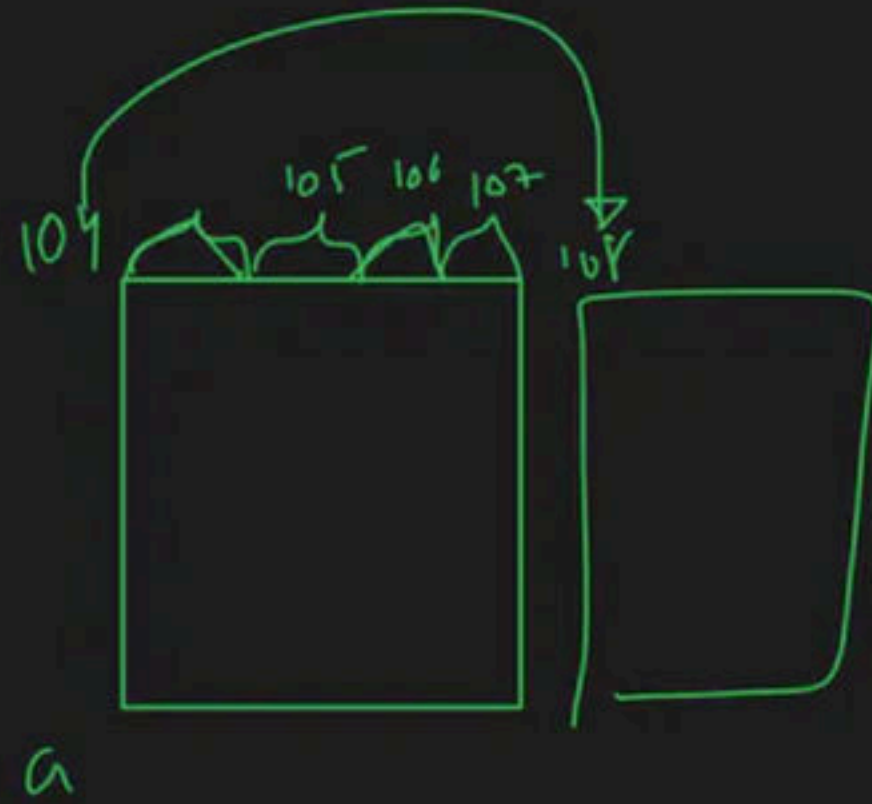


ptr



104 + 1
= 105

15 sec →



Pointer Arithmetics

⇒ arr ⇒

int a[5] = {1, 2, 3, 4, 5}

int *b = a;

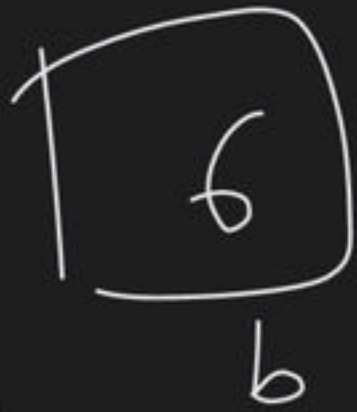
void *c = (void *)a;

int b = 5;
int *a = &b;

↓
*a = &a + 1 *
↓
= 5 + 1

cout << *(&b + 1) ??

cout << *(c + 1) ??
=

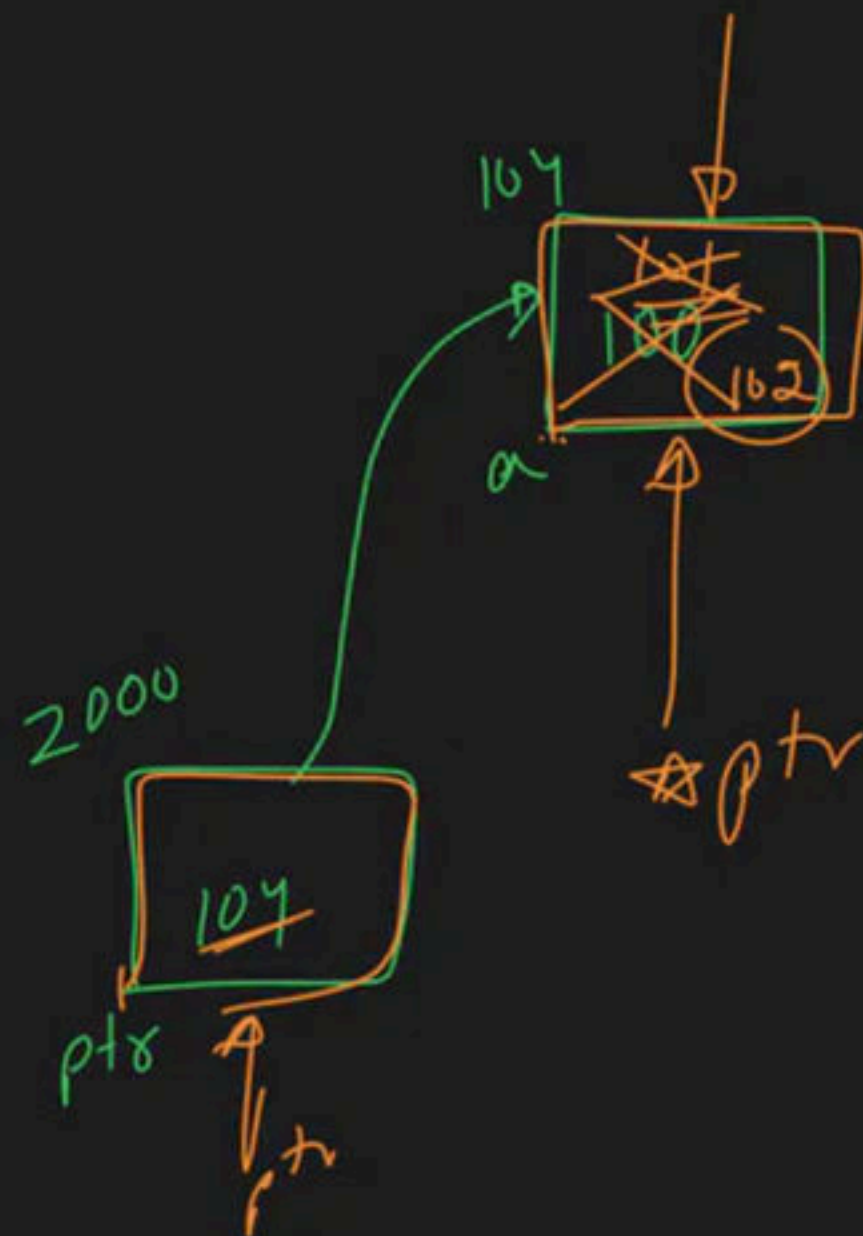
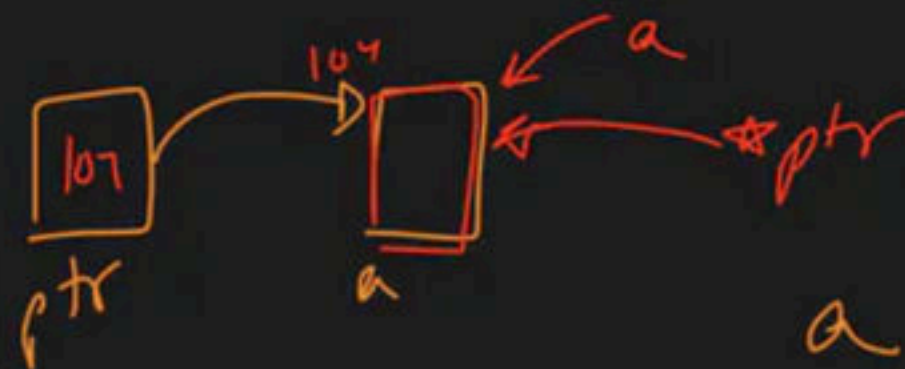



```
int a = 100
int * ptr = &a
```

$a = a + 1$

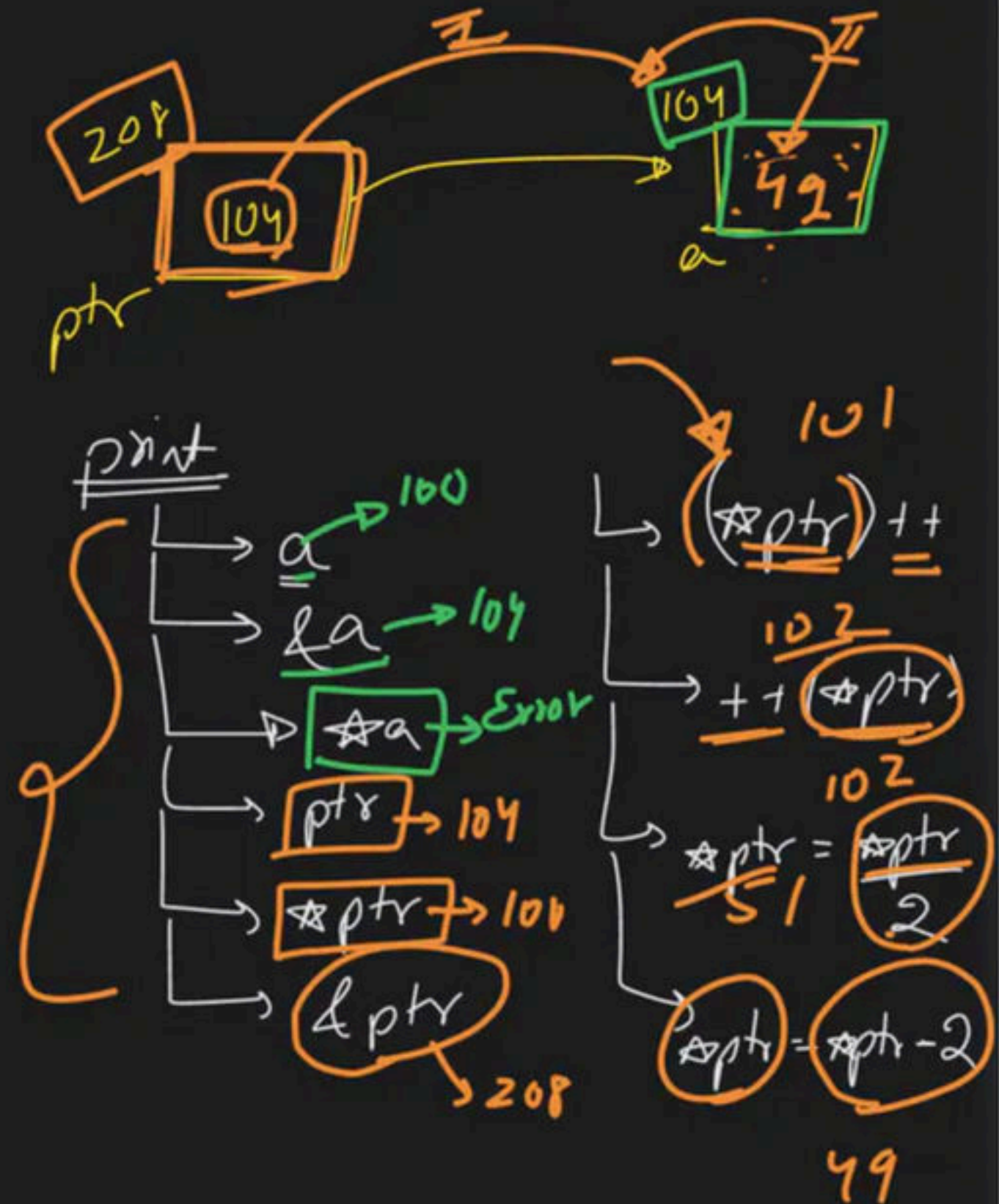
$\star ptr = \star ptr + 1$
~~100~~ 101 + 1

Value present at
 address stored in
 ptr




```
int a = 100;
int *ptr = &a;
```

1 minute → ?



```

int a = 5;
int * p = &a;
int * q = p;

```

pointer copy

a	→	5
&a	→	1008
*a	→	Error
p	→	1008
&p	→	216
*p	→	5
q	→	1008
&q	→	318
*q	→	5



2 min

~~int *q = *p;~~

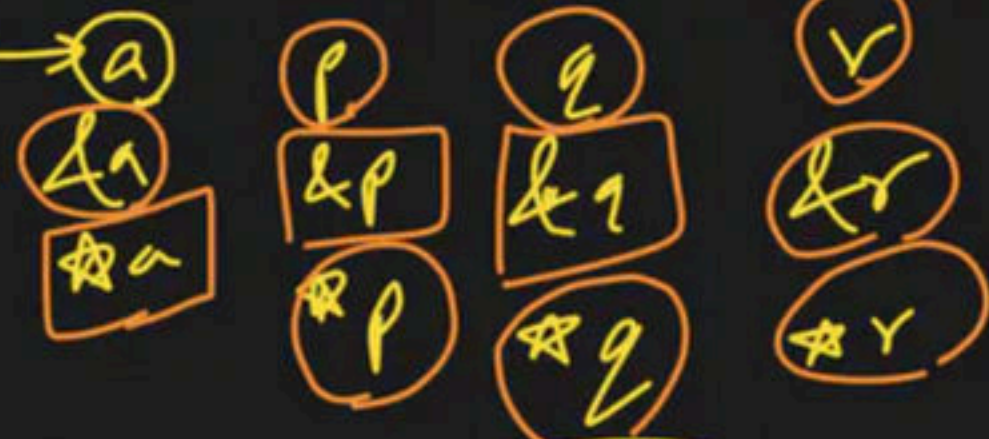
g.d.1.

☆☆

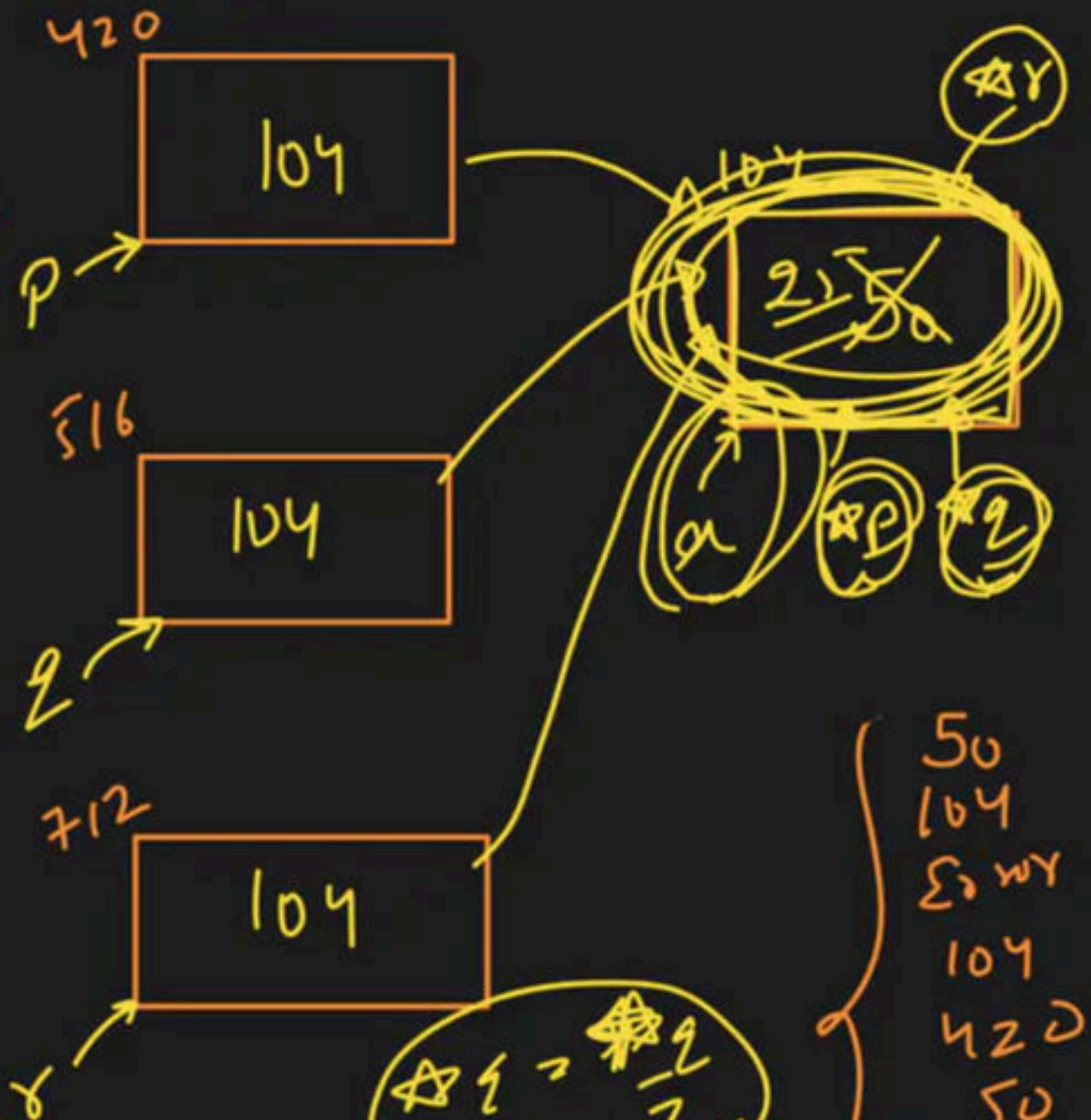
$\star p = \star q$

$a = 50$

```
int a = 50;
int *p = &a;
int *q = p;
int *r = q;
```



$\star q = \star r$



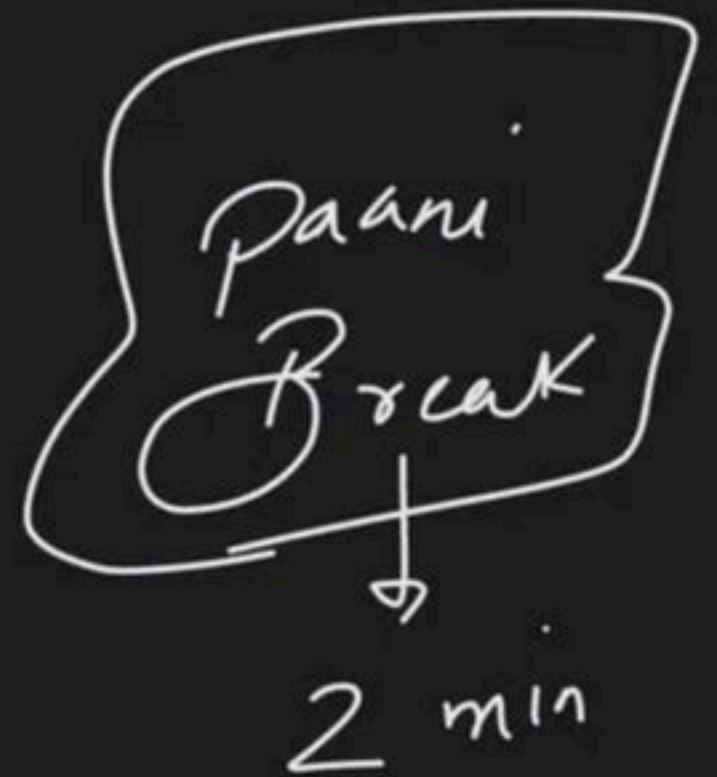
$\star p = \star q$

$a = 50$

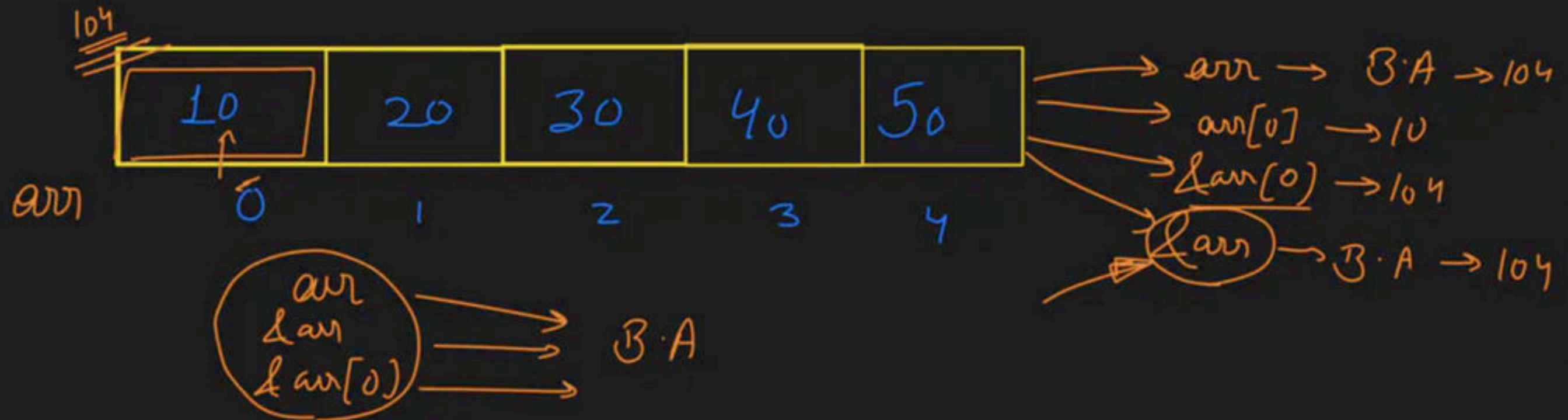
$\star r = \star q$

50
104
516
104
712
50
104
516
50
104
712
50

→ Pointer with Array



int arr[5] = { 10, 20, 30, 40, 50 }



arr
arr[0] → BA

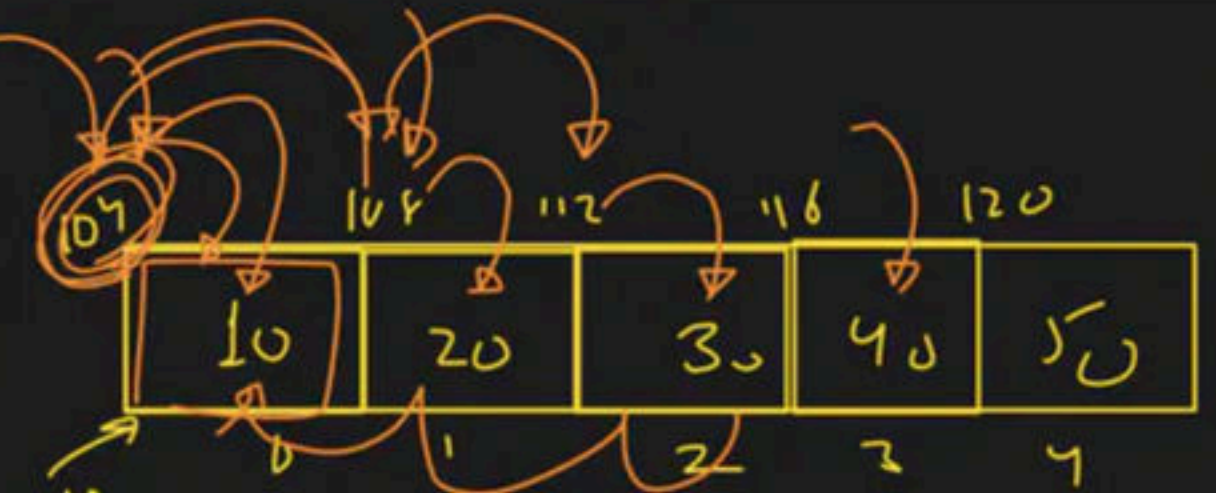
2 min

int arr[5] = {10, 20, 30, 40, 50}

→ print

arr → 104
 &arr → 104
 arr[0] → 10
 &arr[0] → 104
 *arr → 10
 *arr + 1
 10 + 1 = 11

*arr → 104
 *(arr + 1) → 11
 *(arr + 2) → 104 + 2
 *(arr + 3) → 104 + 3



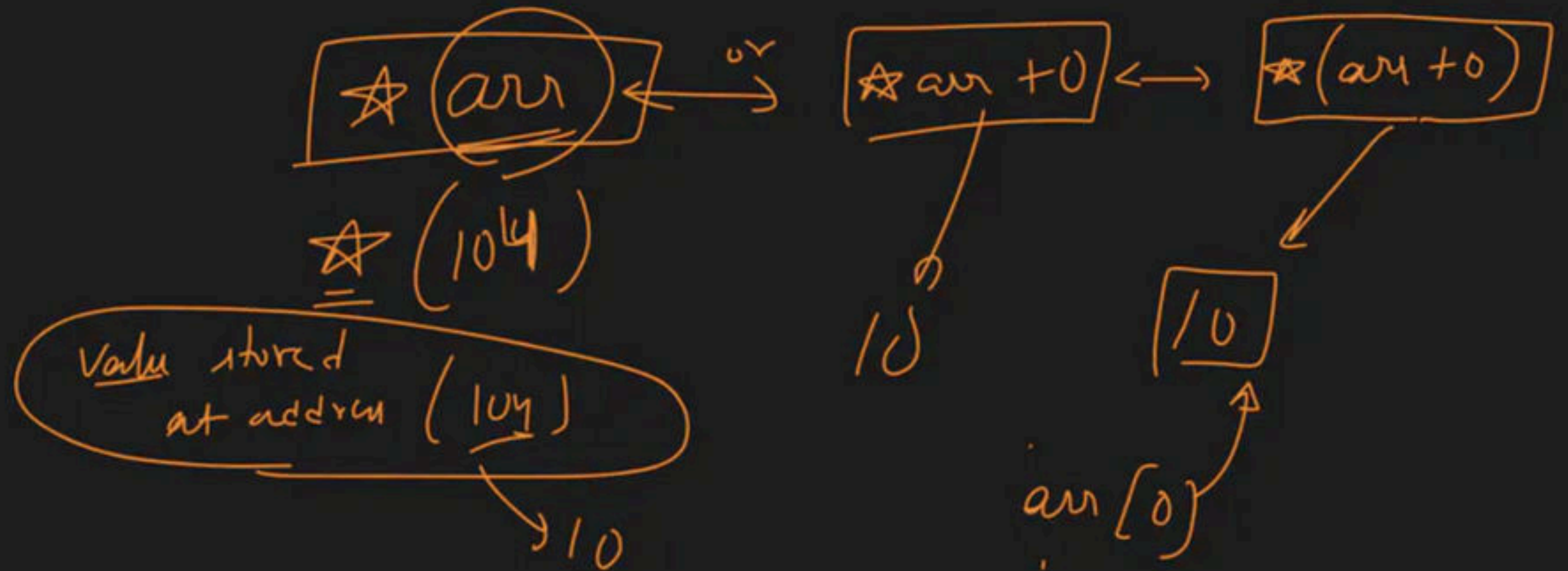
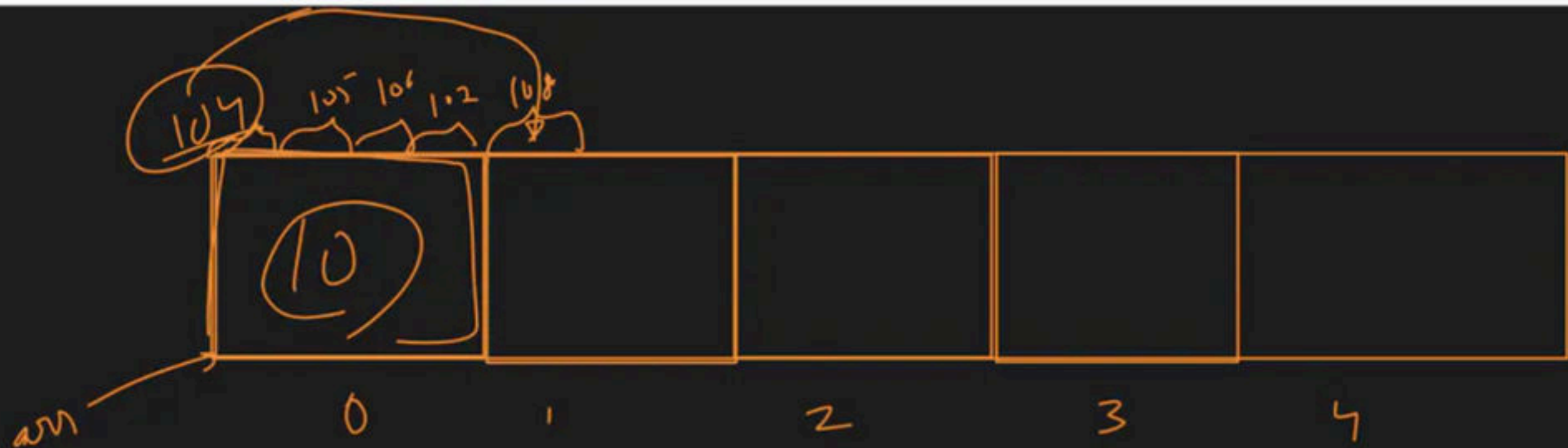
$$104 + 1 = 108$$

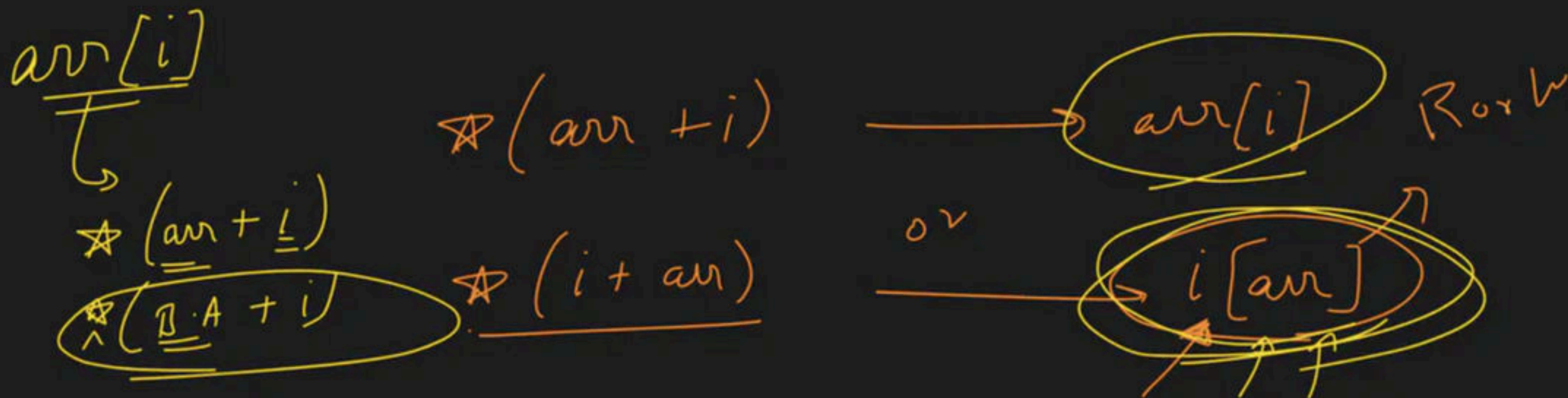
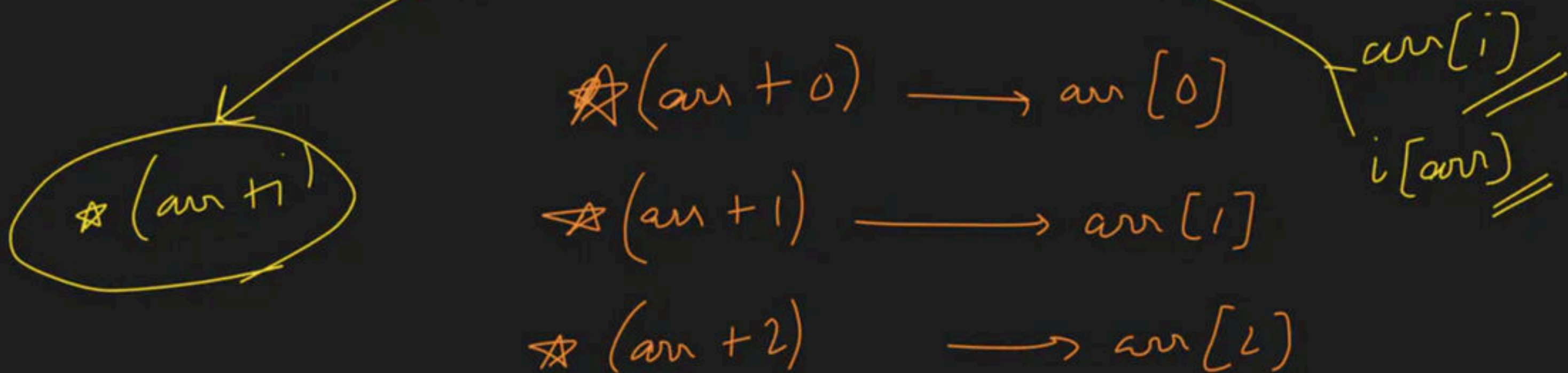
20
 104 + 2 → 112 → 30
 104 + 3 → 116 → 40

104
 104
 10
 104
 10
 11

11
~~11~~ 20
 30
 40

arr[0]





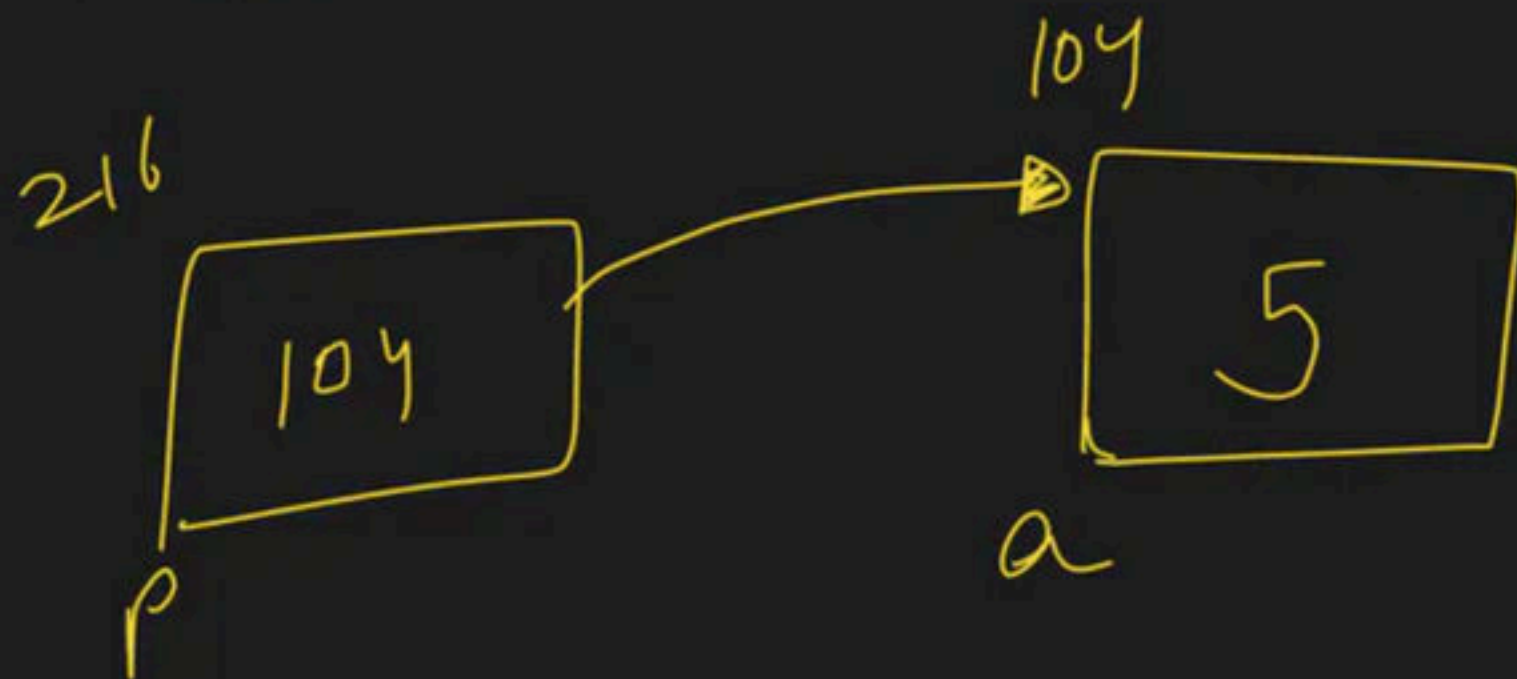
$\star(arr + i) \rightarrow arr[i]$

Note:

int a = 5;

int *p = &a

p = p + 1



int a[5] = { 1, 2, 3, 4, 5 }

Constant pointer

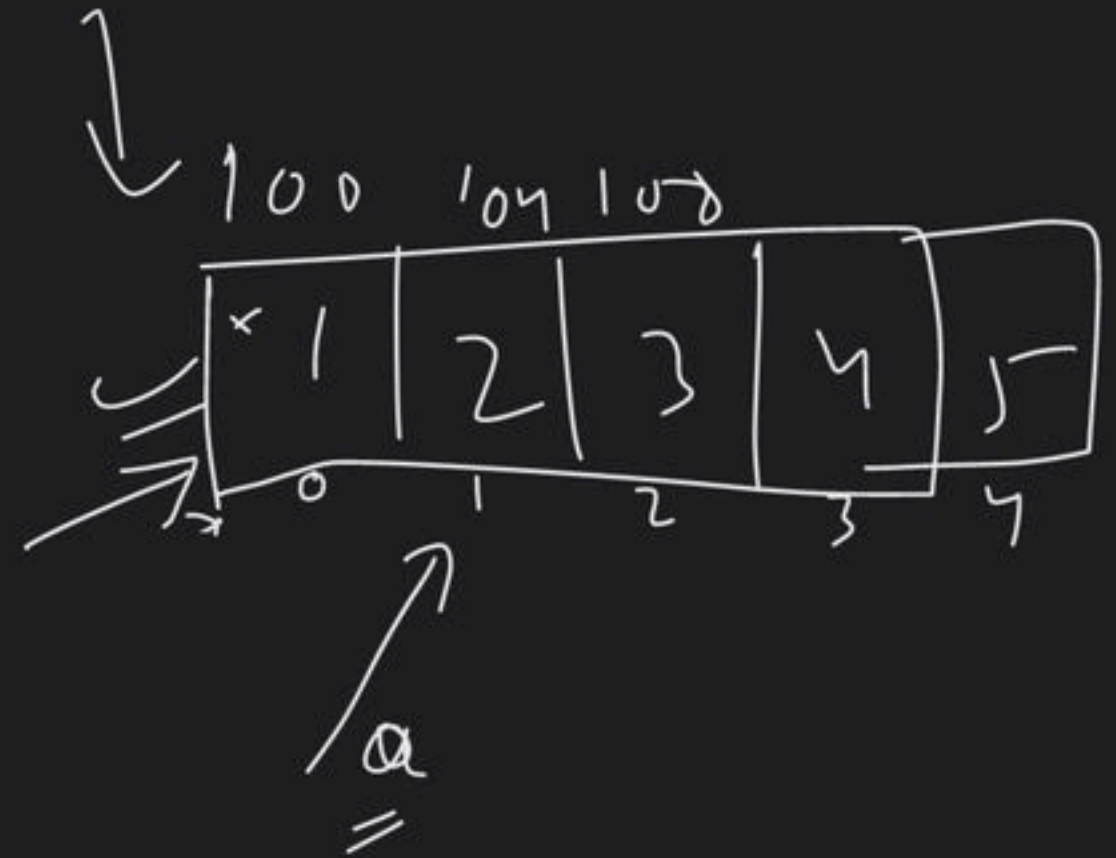
$a = a + 1$? ?
=



$a = a + 1$

$a = 104$

$a[-1]$



$a = 100$

int arr[5] = {10, 20, 30, 4, 50}

arr = arr + 1

Why not possible in this case →

~~HW~~

2 min

→ int arr[4] = {10, 20, 30, 40}

int *p = arr ;

int *q = arr + 1 ;

arr
4arr
arr[0]
&arr[0]

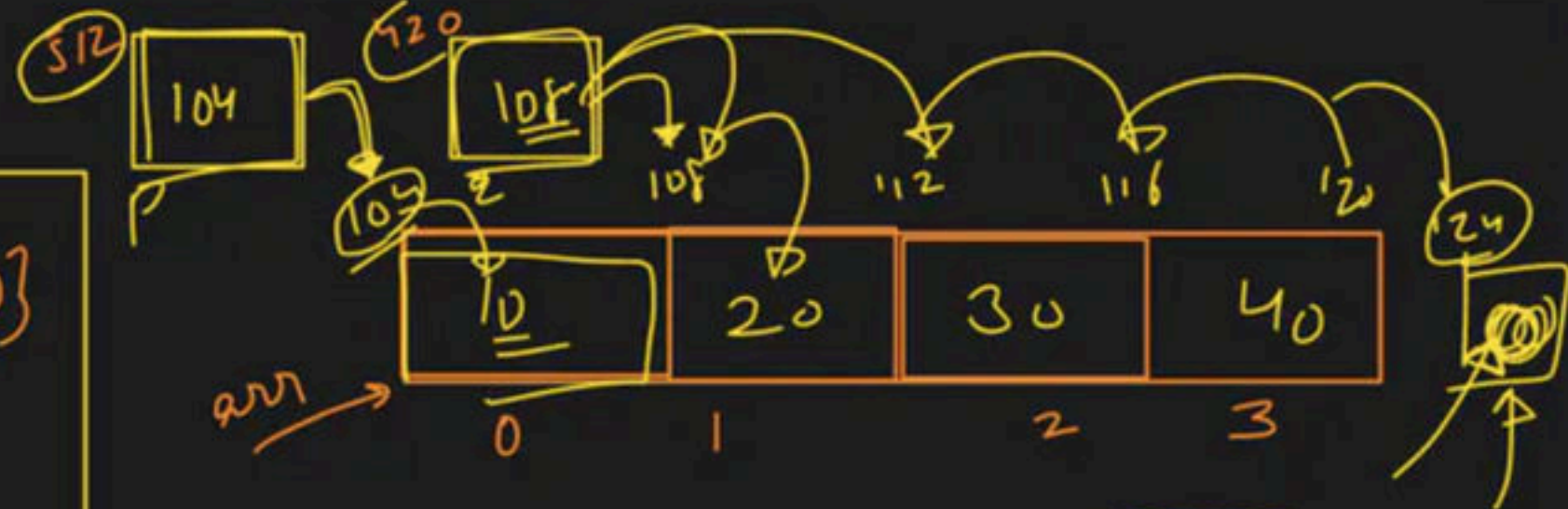
p
&p
*p
q
&q
*q

*p + 1 → 11

*(p) + 2

*(q) + 2

*(q + 4)



11
10 + 2 = 12
20 + 2 = 22

104
104
10
104
104
512
→ 10
108
420
→ 20

int arr[4] = {10, 20, 30, 40}

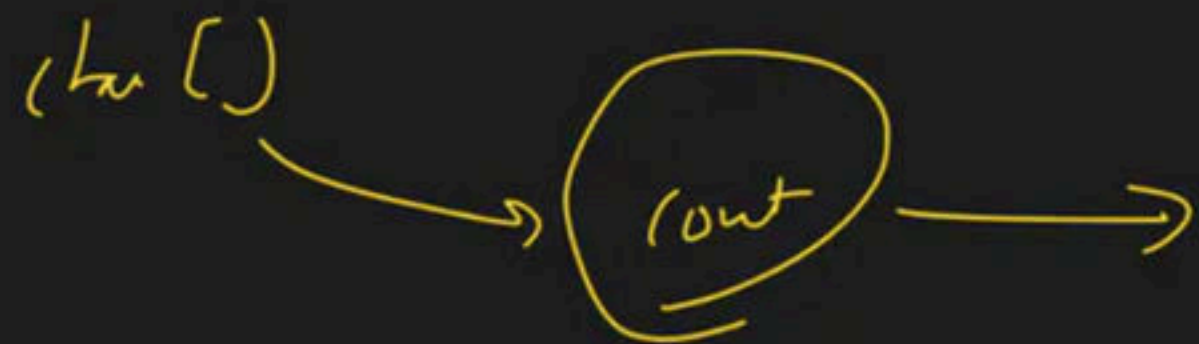
sizeof(arr) → 4 × 4 = 16

int *p = arr

sizeof(p)



8



→ cptr[0]

char ch[50] = "love"

char * cptr = ch

~~&ch~~

love

cout << cptr;

love

love

love

love

cptr

int arr[10] = {10, 20, 30}

int * p = arr

104

cout << p; → 104

104

104
arr

*cptr

*cptr
*(cptr+0)

cptr[0] →

cout << ch

char ch[50] = "love";

char * ptr = ch

ptr is a pointer
to a char array

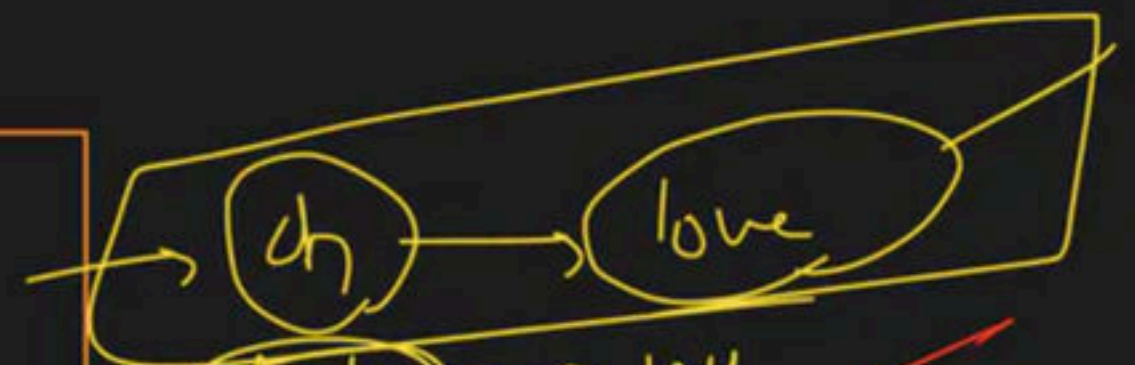
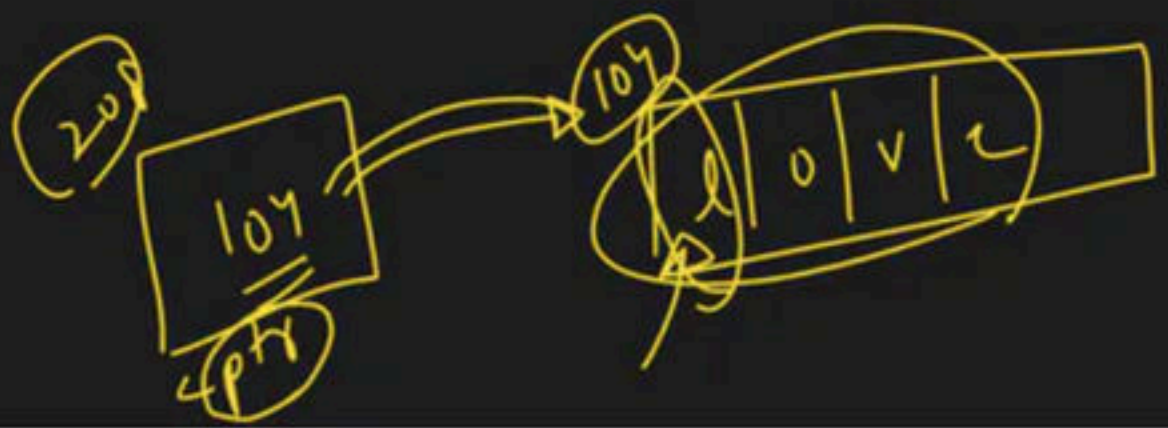
~~ch~~

&ch

~~&ch[0]~~

ch
&ch

2 min



~~&ch~~ → 104

→ ch[0] → l

→ &ptr → 208

→ *ptr → *(ptr + 0)
 ↳ ptr[0]

→ ptr → love

auto a = b;



auto a = (void*) b;

void*

char * ptr = "codehelp"; valid ✓✓



constant

↳ stack

⇒ ptr[5] = 'd';



~~under~~
error

$\Rightarrow$

codehelp



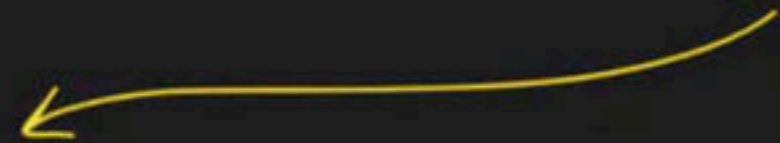
Bottom Right



facing an
Issue



check



face an issue



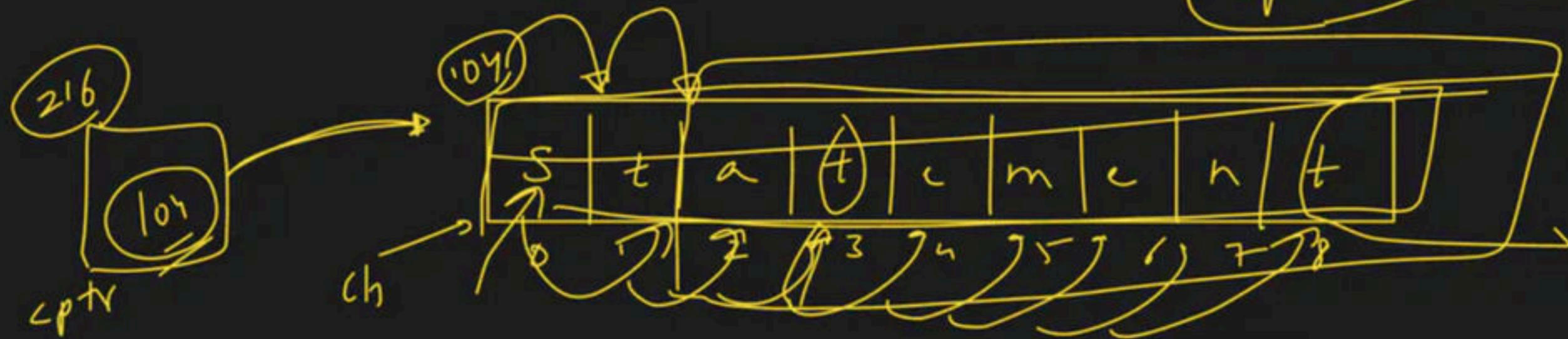
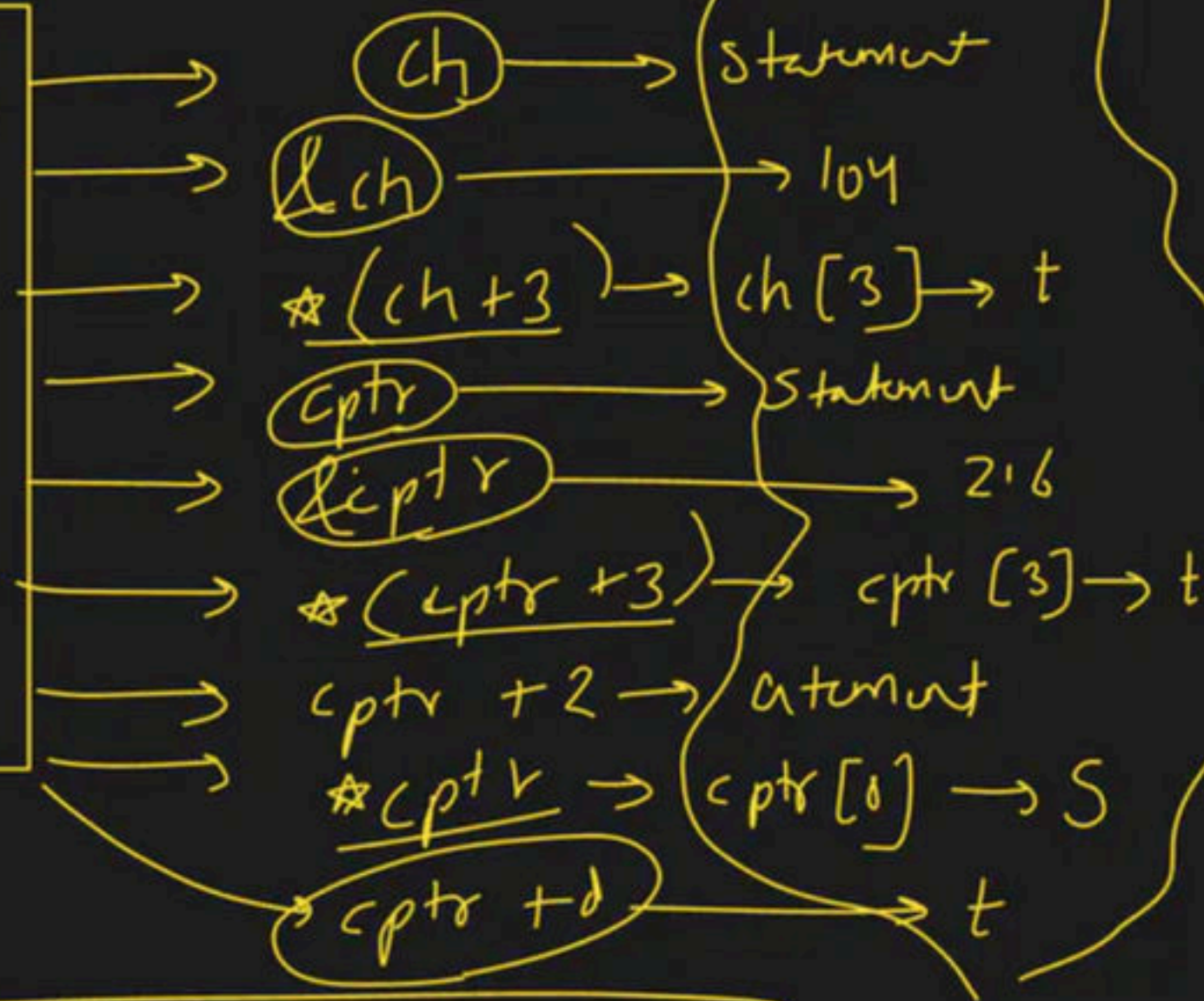
fill



Submit

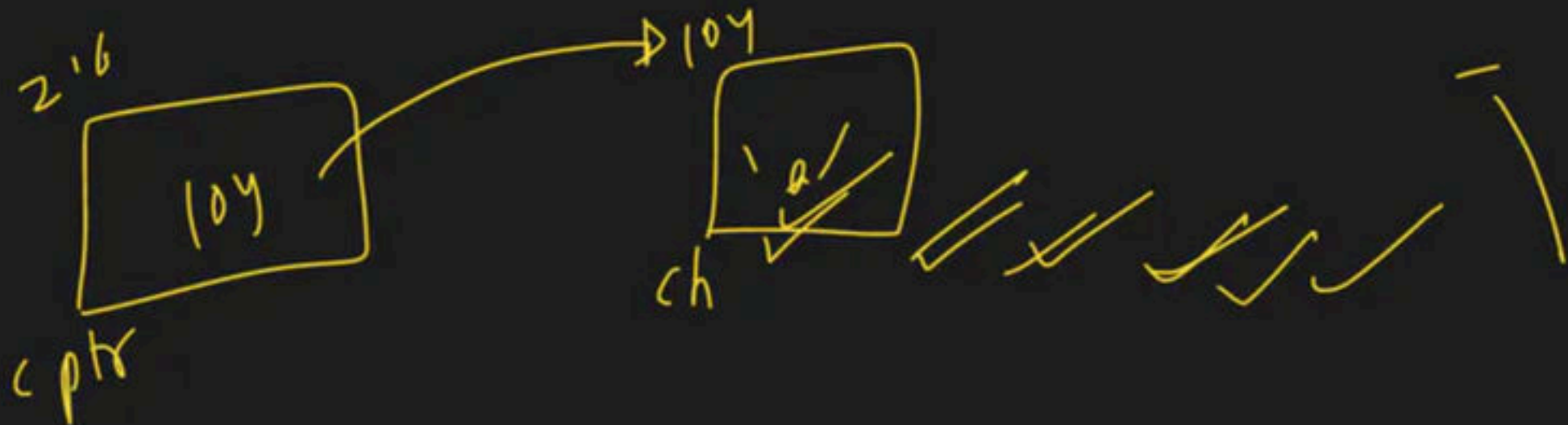
2 min

```
char ch[30] = "Statement";  
char * (cptr) = &ch[0];
```



`char ch = 'a';`
`char * cptr = &ch;`
`cout << cptr`

→ cptr
↓
a



char ch[10] = "Babbar"

char *c = ch

~~3rd~~
practice.

why?

char *cptr = "Babbar"

~~low~~ < cptr

possible

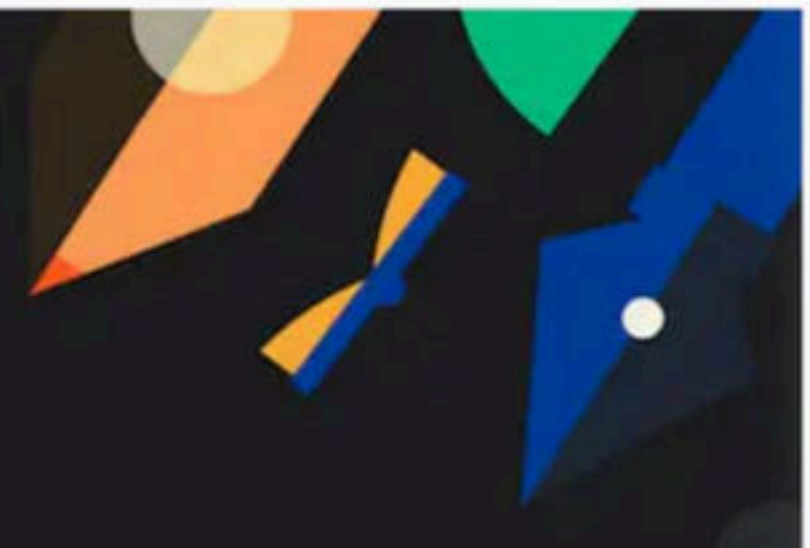
~~temp storage~~

S L I E T

L1D

Low Level
Design

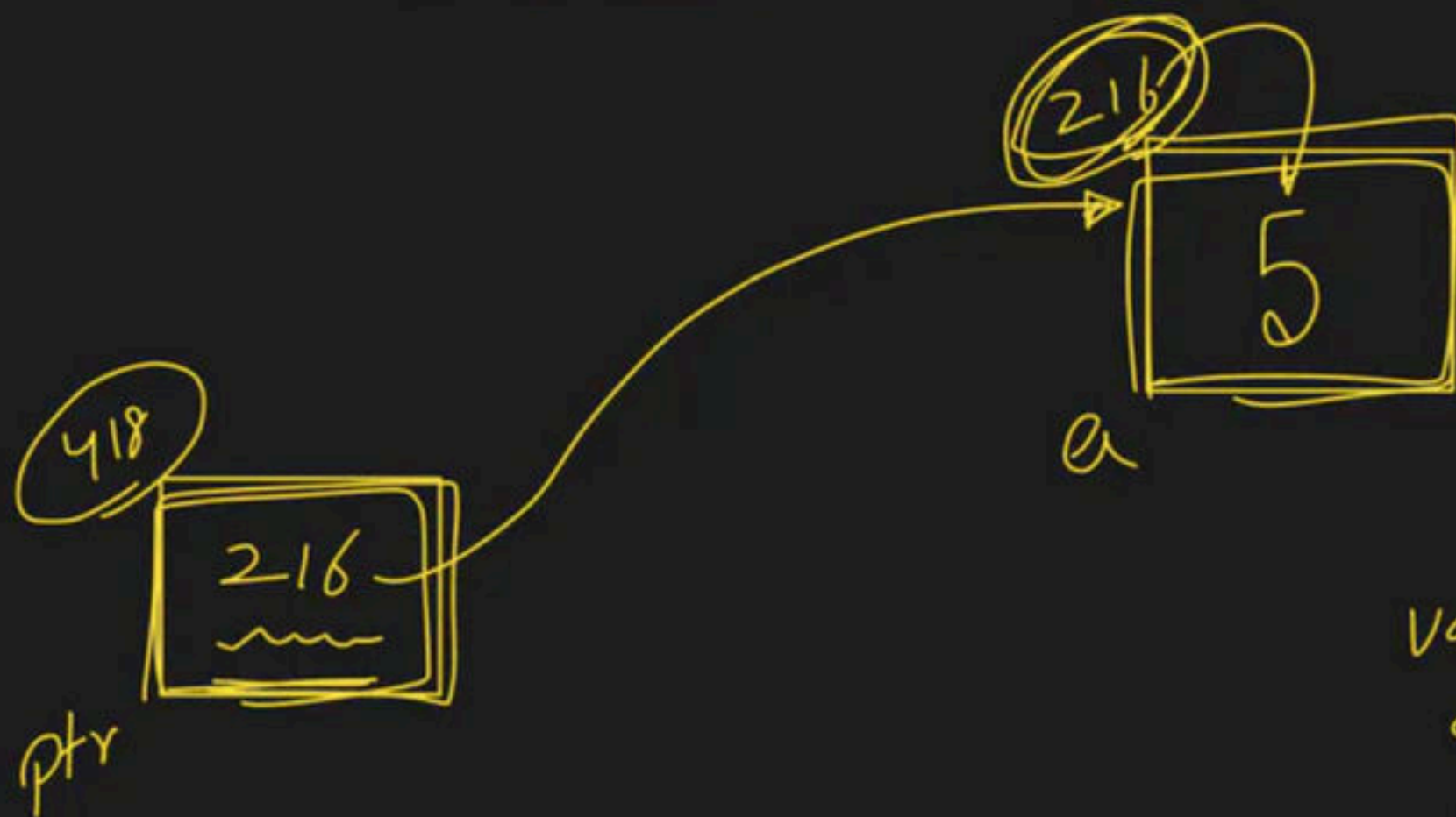
after 26th Oct



Pointers - Class 2

Special class

int a = 5;
int * ptr = &a



a → 5
&a → 216
ptr → 216
*ptr = 5
&ptr → 418

Value stored
at location
~~ptr~~ 216

char ch[10] = {'a', 'b'}

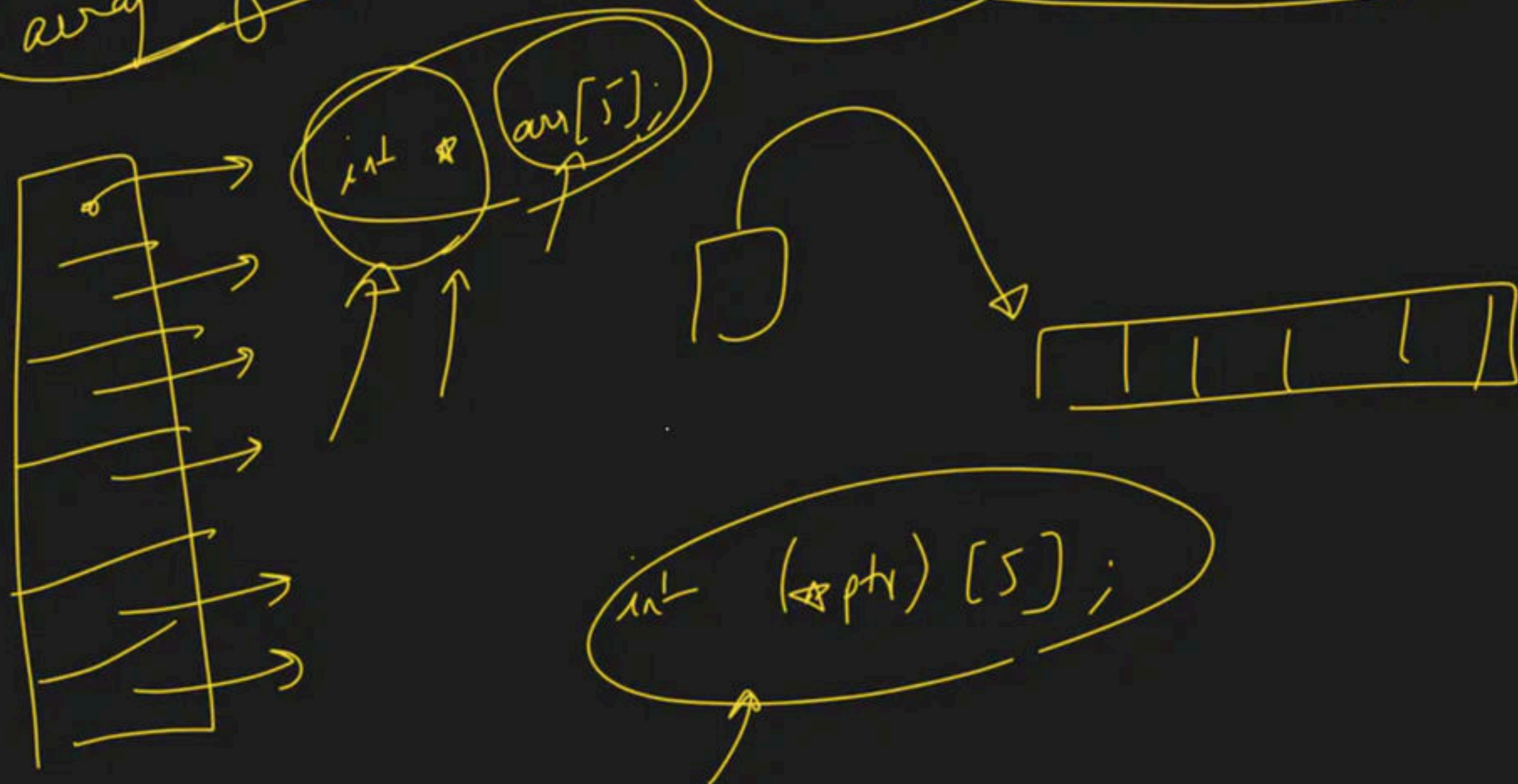
char * ptr = &ch



integer array

array is ~~point~~

Pointer to an array



~~int~~ ptr = &a; ✓✓

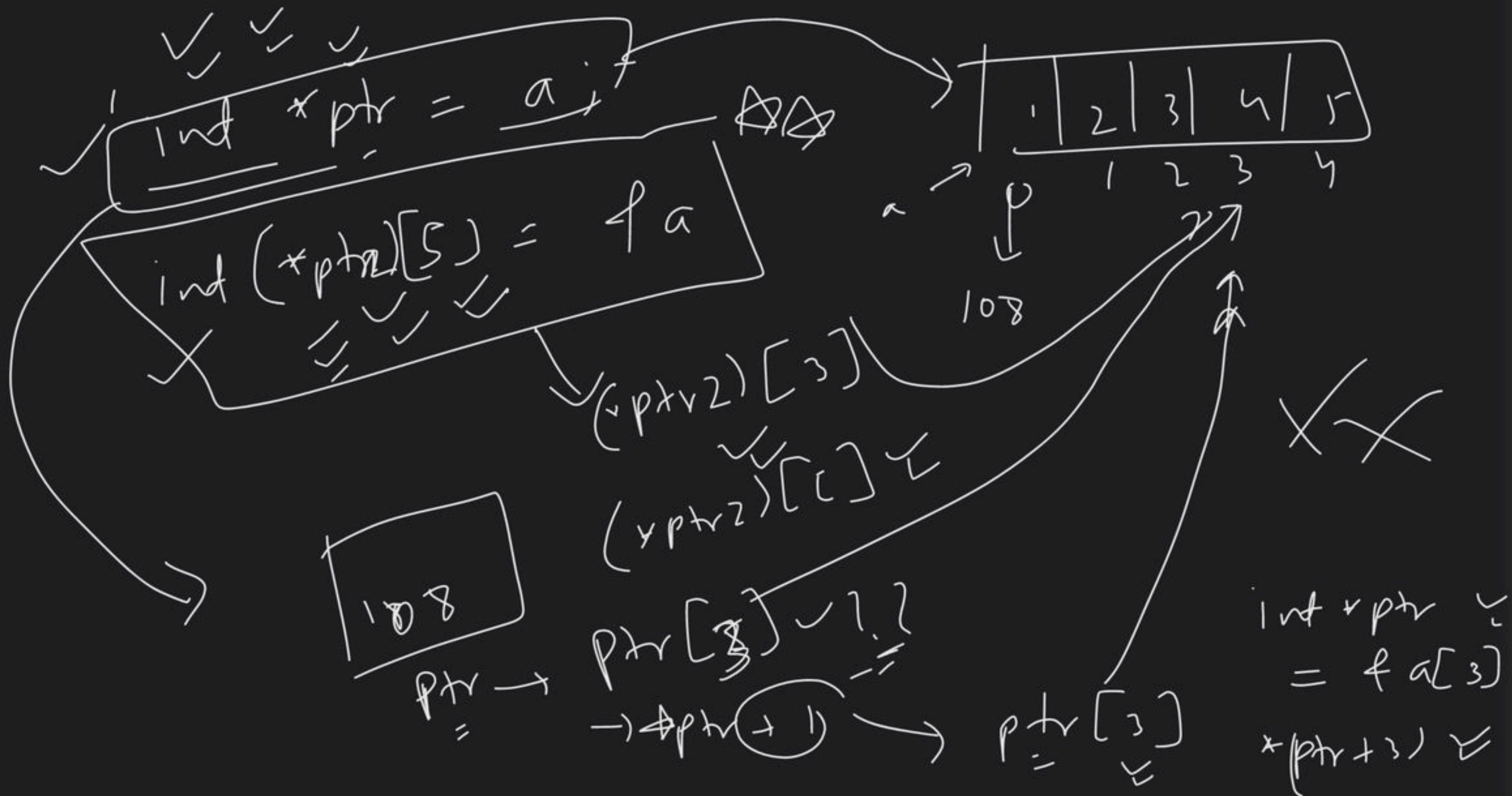
Print $\Rightarrow$ 4 $\Rightarrow$ B A

Print $\rightarrow$ a $\Rightarrow$ B A

$$\text{ind}^* \text{ptr} = \wedge' \quad \checkmark \checkmark$$

Pointer to
an integer.

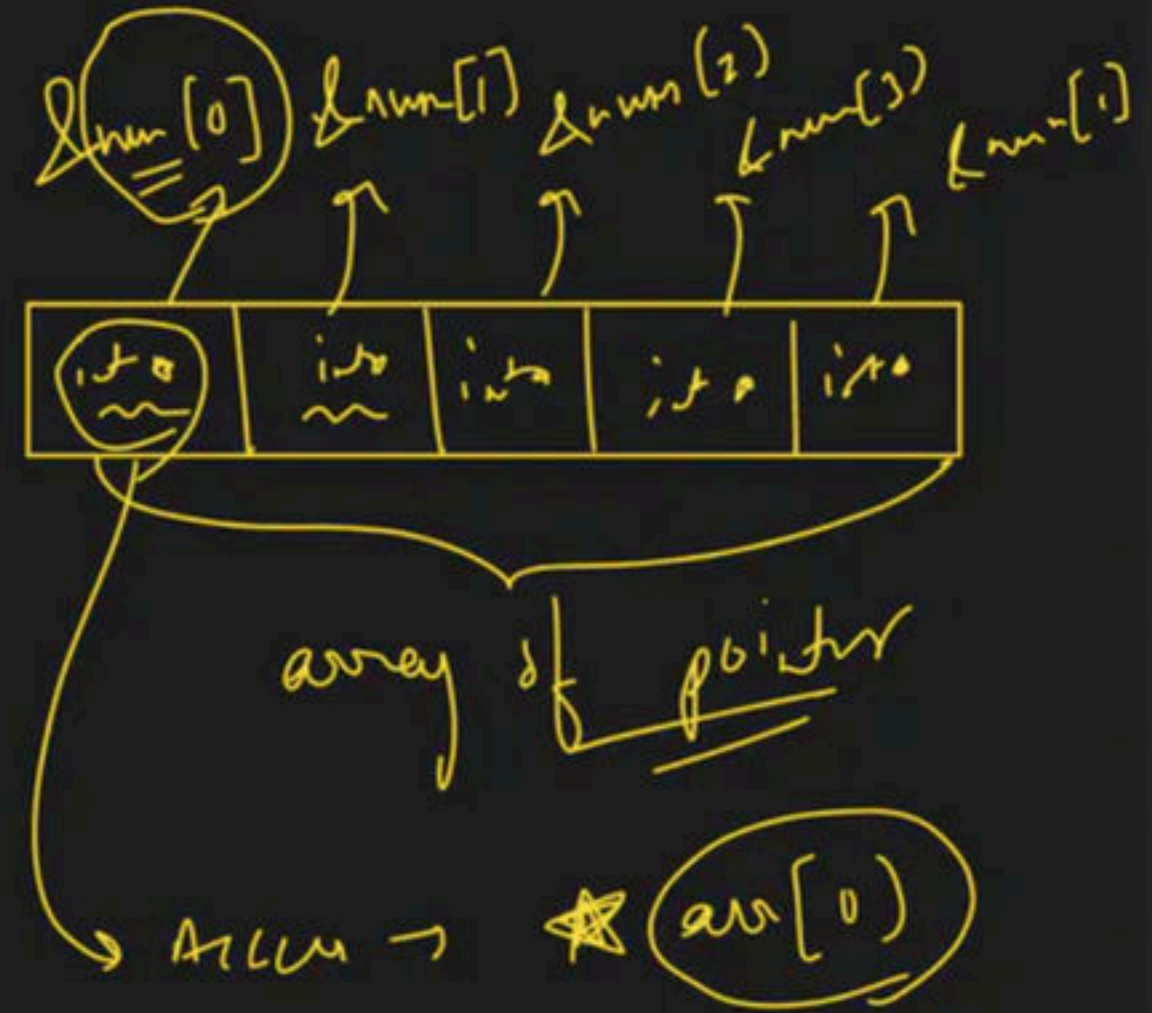
$\Rightarrow$ `int (*ptr)[5]` = $\&a \Rightarrow$ pointer to an array



$$* (ptr + i) == \underline{\underline{ptr[i]}}$$

int arr[5]

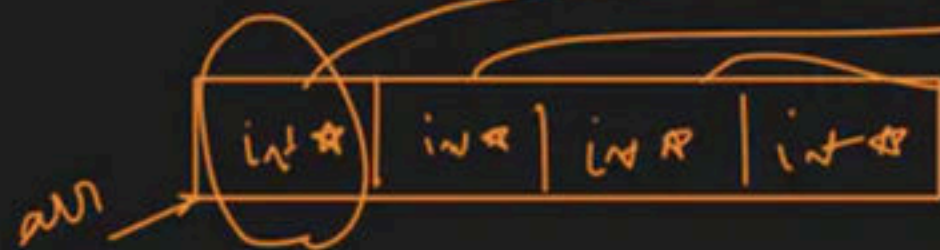
int * arr[5]



→ Array of pointer

→ int * arr[4]

num[] = {1, 2, 3, 4}



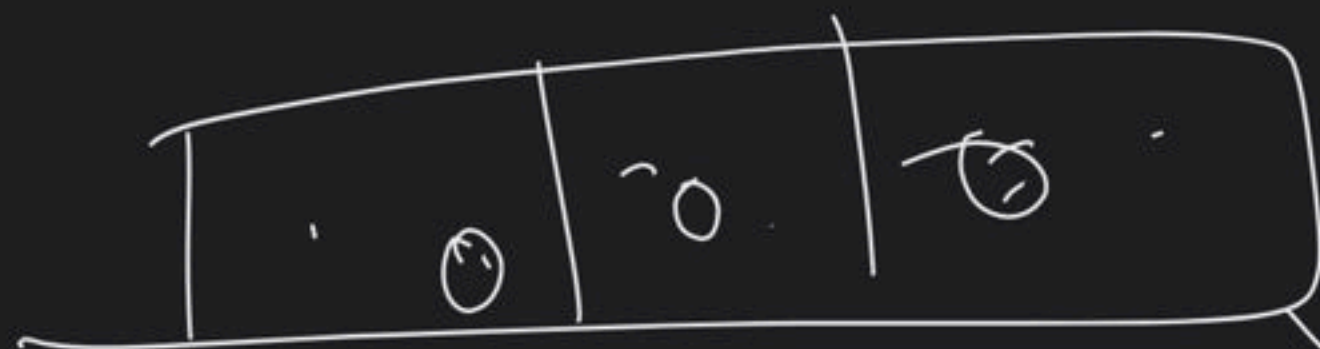
→

arr[0]	→	&num[0]
arr[1]	→	&num[1]
arr[2]	→	&num[2]

→ Access →

• *arr[0]
*arr[1]
*arr[2]

~~&arr~~
~~arr[0]~~
~~arr~~



int a = 1 → 104

int b = 2 → 308

int c = 3 → 720

int* ptrarray[3] = {0}

ptrarray[0] = &a, ✓

" [1] = &b

[2] = &c;

pointer to an array

doubt $\rightarrow$ detailed
 $\hookrightarrow$ pointer type doubt

$\text{int} * \text{ptr} = \boxed{\text{arr}}$

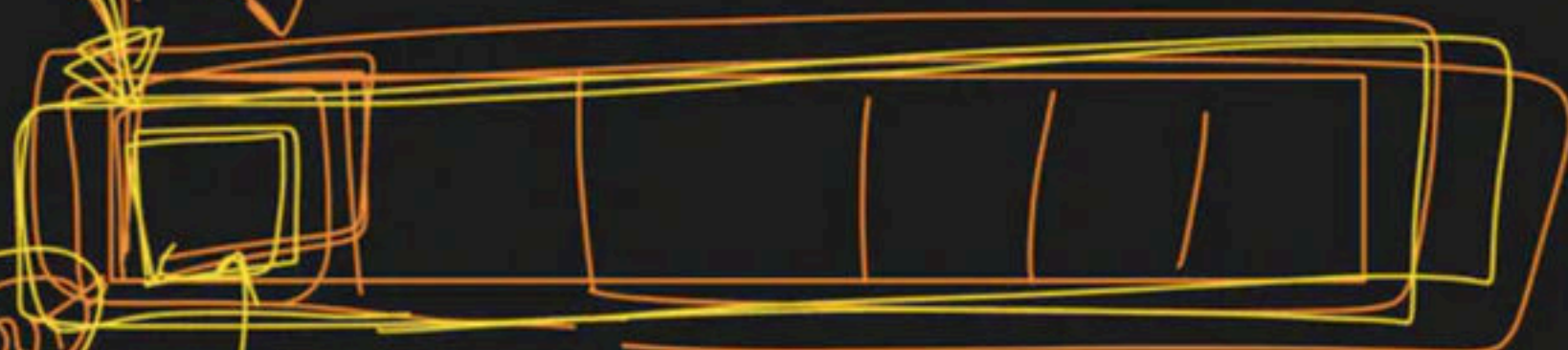
$\text{int} * \text{ptr} = \text{arr}$

ptr

$= \text{arr}$

arr

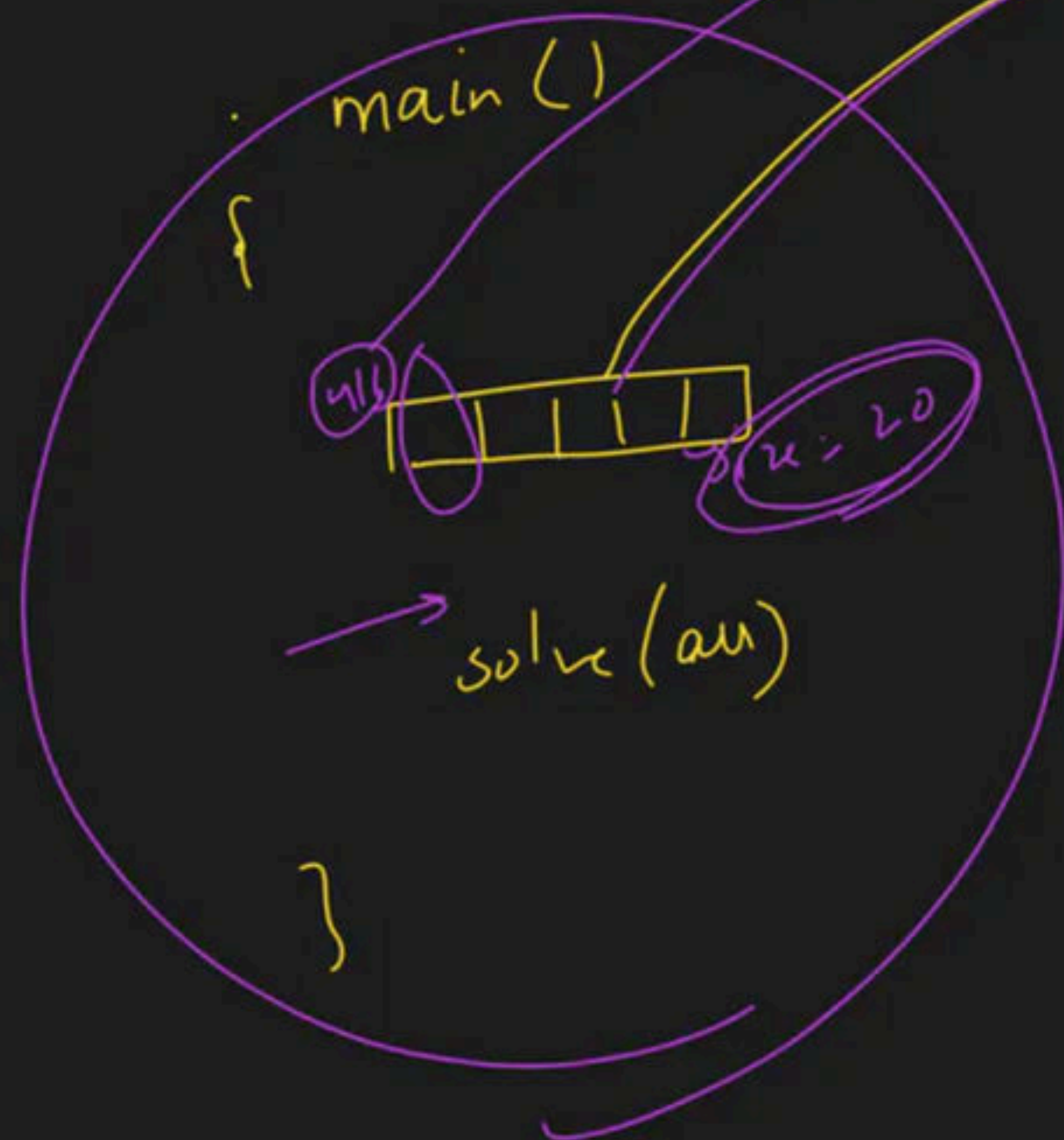
$\boxed{\text{arr}}$



$\text{in } (*\text{ptr})[5]$
 $\uparrow$
 $= \text{arr}$

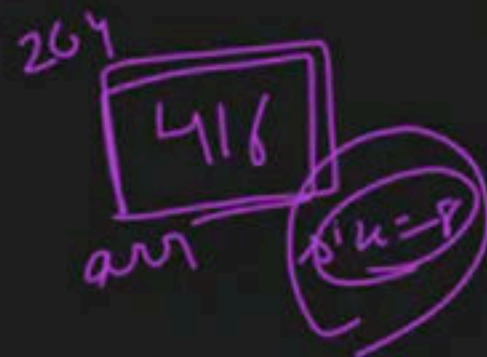
~~$\text{int} * \text{ptr} = \text{arr}$~~

→ Pointer → functions



~~solve~~
{

~~(int arr[])~~
~~(int arr)~~ → int arr



→ arr → 416
→ arr → arr[0]

}

main()

{

int arr[3] = {1, 2, 3}

solve(arr, 3)

solve(int arr[], int siz.)

{

int *arr =

arr[3] =

3

siz

216

104

arr

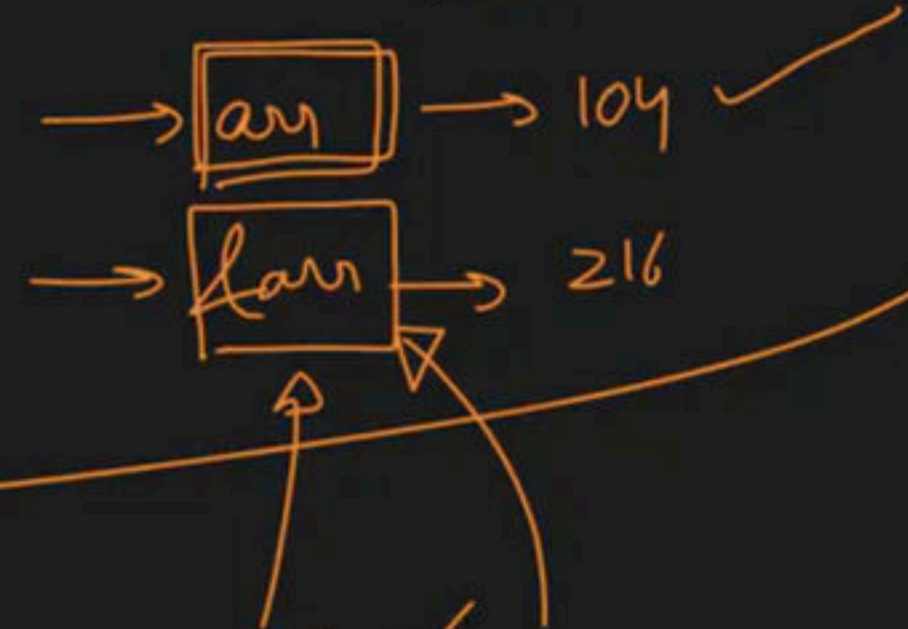
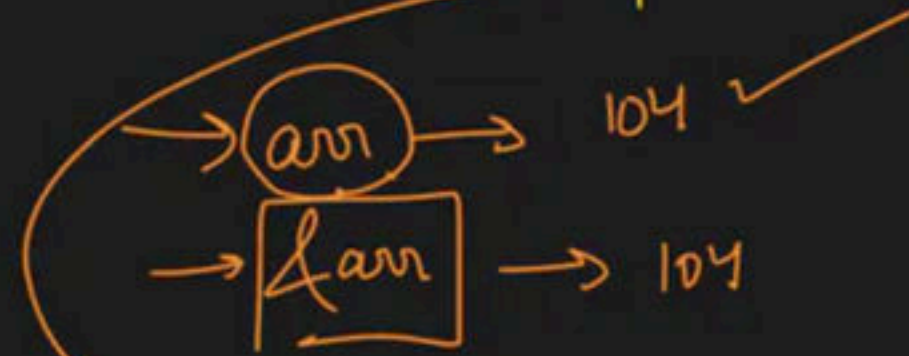
→ arr → 104 ✓

→ &arr → 216 ✓

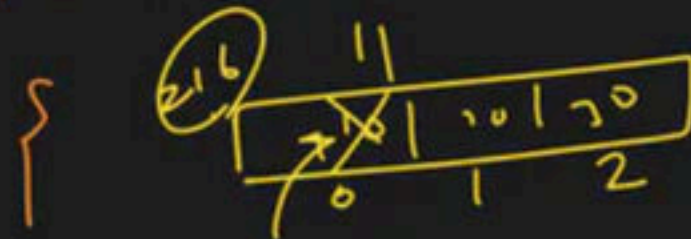
diff ✓

}

}



main ()



arr[] = { 10, 20, 30 }

solu (arr)

// init array

11, 20, 30

solu (int arr[])

{
arr

*arr = *arr + 1
= 10 + 1
= 11

→ Pointer to Pointer

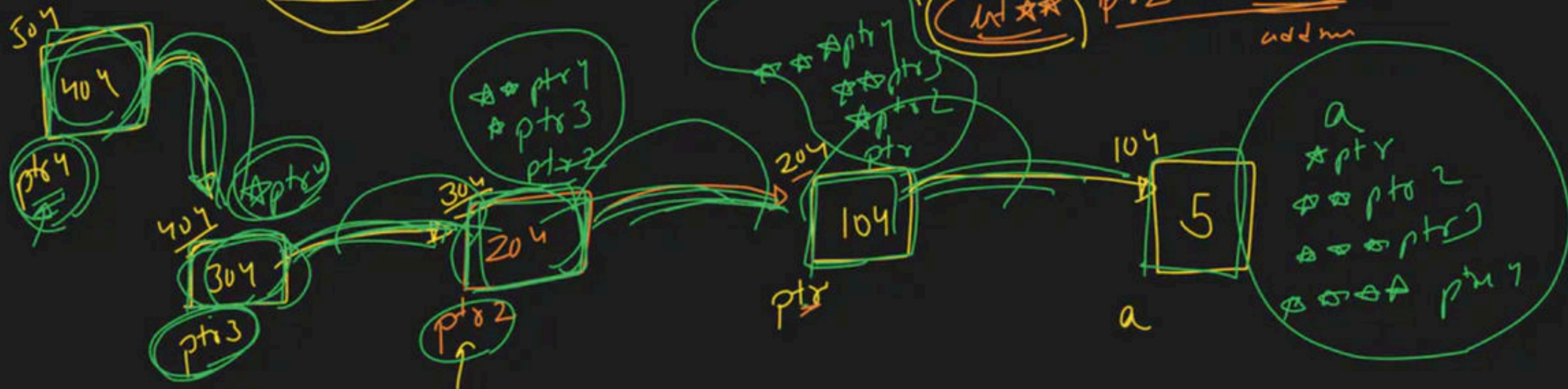
`int **** ptr4 = &ptr3`

`int *** ptr3 = &ptr2`

`int a = 5`

`int * ptr = &a`
address

`int ** ptr2 = &ptr`
address



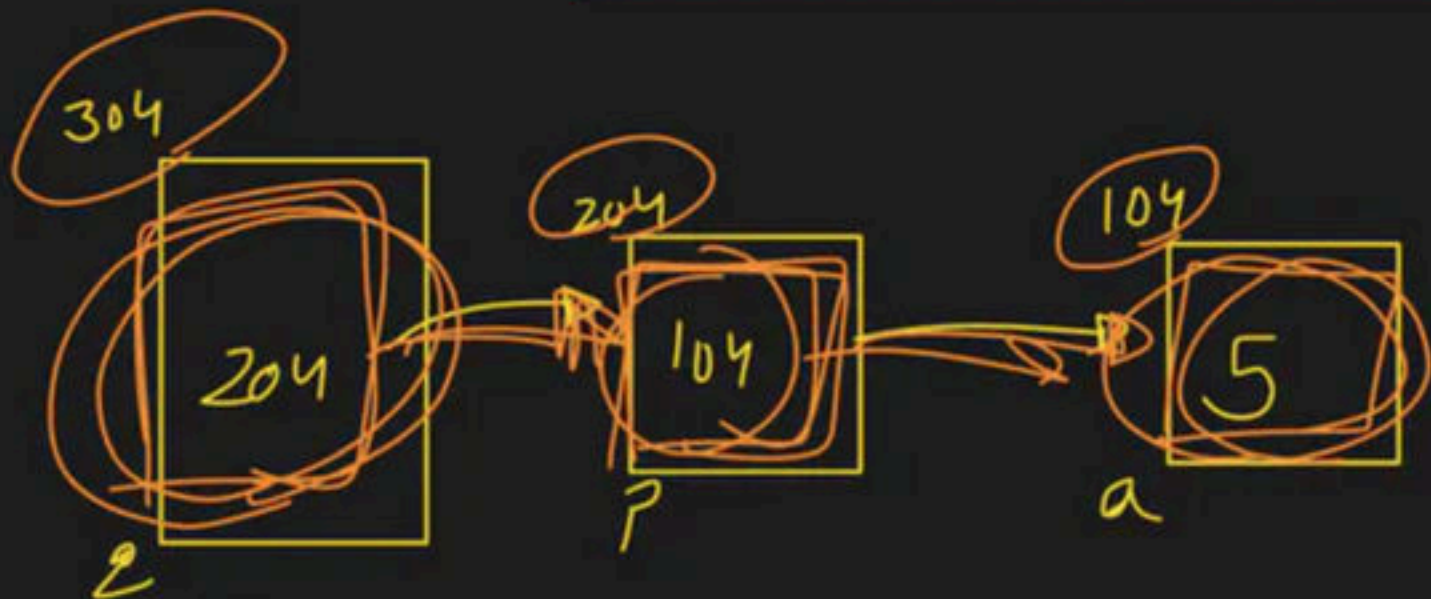
Qua;

(2 min)

```
int a = 5; ✓  
int *p = &a;  
int **q = &p
```

ptr ↓

→ a → 5
→ &a → 107
→ p → 104
→ &p → 204
→ *p → 5
→ &q → 204
→ &q → 307
→ *q → 104
→ **q → 5



pendiy

main ()

{

int a = 5;
int *p = &a;

cout << p →
<< &p →
<< *p

104
204
5

solve (p)

p 1
*p 1

solve (int *p)

{

*p = *p + 5

}

$\Rightarrow$ int $n = 10;$

int * ptr = &n;

cout << *ptr + 2 * *ptr - 3 * *ptr;

??
=

$\Rightarrow$ error X

10 + 2 * 10 - 3 * 10

2) 10 + 20 - 30

30 - 30 = 0

a = 5
cout << 2a

2 + a

(2a) X

⇒

$x = 5, y = 10, z = 15;$

$\text{int } *arr[] = \{x, y, z\};$ ✓✓

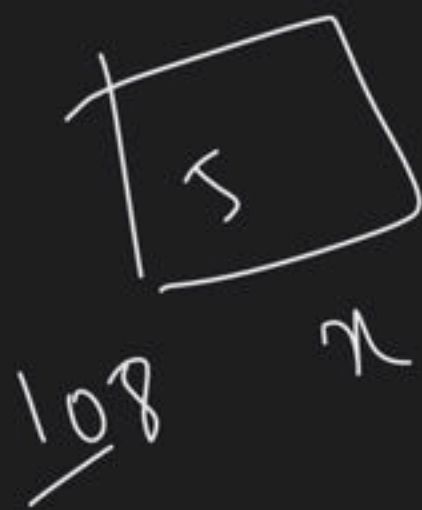
→
=

$(out \ll *arr[i] \ll endl);$

int main ()

x = 5, y = 10;

Swap (&x, &y) → using pointers.



void swap (int *x, int *y)

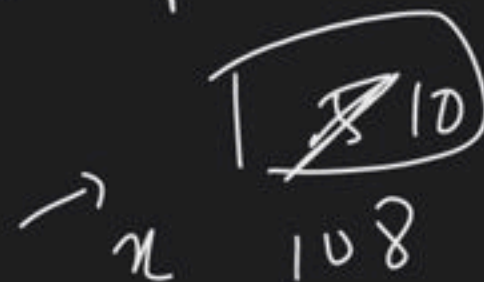
{

int temp = *x;

*x = *y

*y = temp;

swap



int

void swap (int *a, int *b)

swap(4x, 4y)

swap(x, y)

void swap (int 4x, int 4y)

int a = 10;

int *p = &a

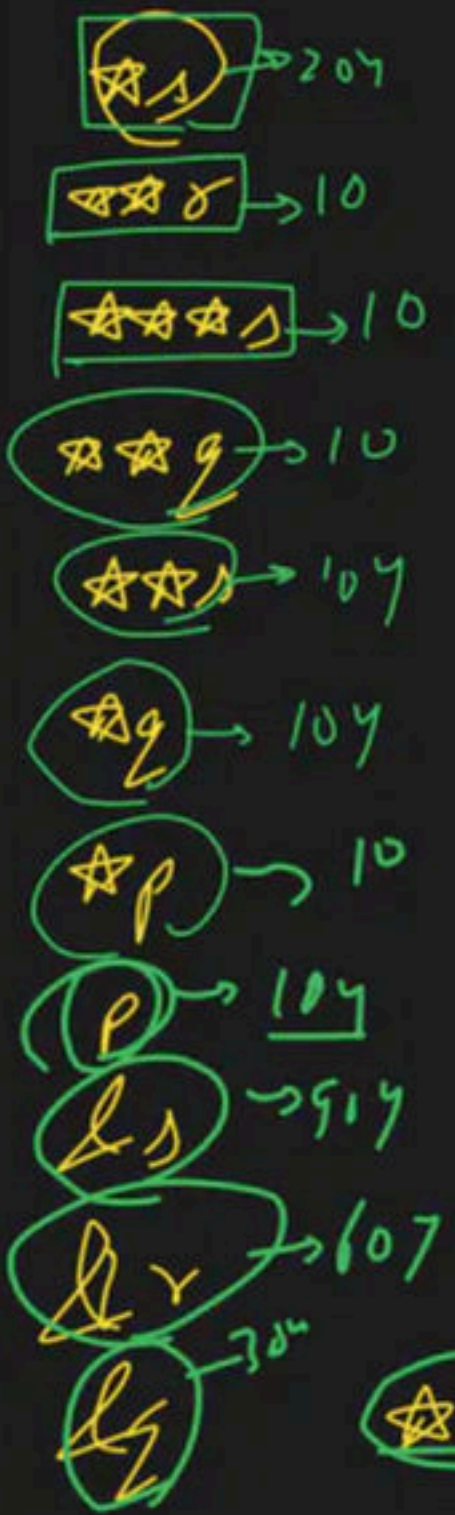
~~int **q = &p;~~

address
of int

*q

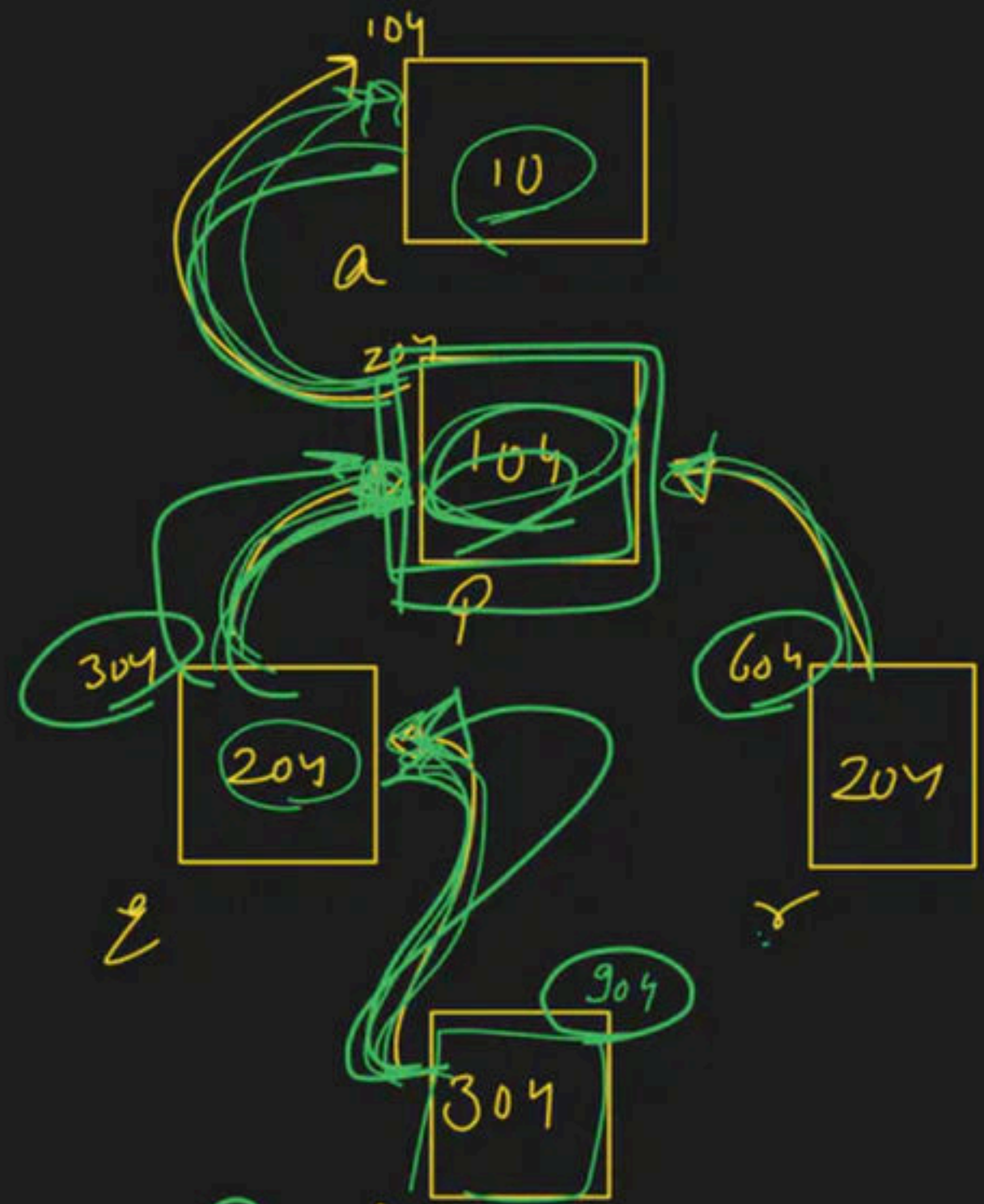
int **q = &p

int *q = &a



```

int a = 10;
int *p = &a
int q = &p
int r = &p
int s = &q;
  
```



1 + 1
 104 + 1 → 105

104 + 1
 104 + 1 * 4 → 104 + 4 → 108

2 min

→ Sun → L.K → 9pm → doubt

→ Next Sat → 9 ~~7~~ ~~part~~ Session

→ Next Week → Mon → Recursion 14

main()

int a = 5;

int *p = &a;

→ p → 104
→ &p → 204
→ *p → 5

solve(p)

→ p → 107 → 104
→ &p → 207 → 204
→ *p → 1 → garbage

Solu

~~int *p~~

int *p
ref

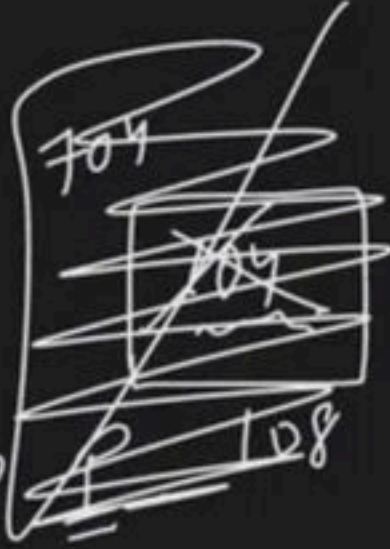
spans by
value

p = *p + 1;
= 104 →

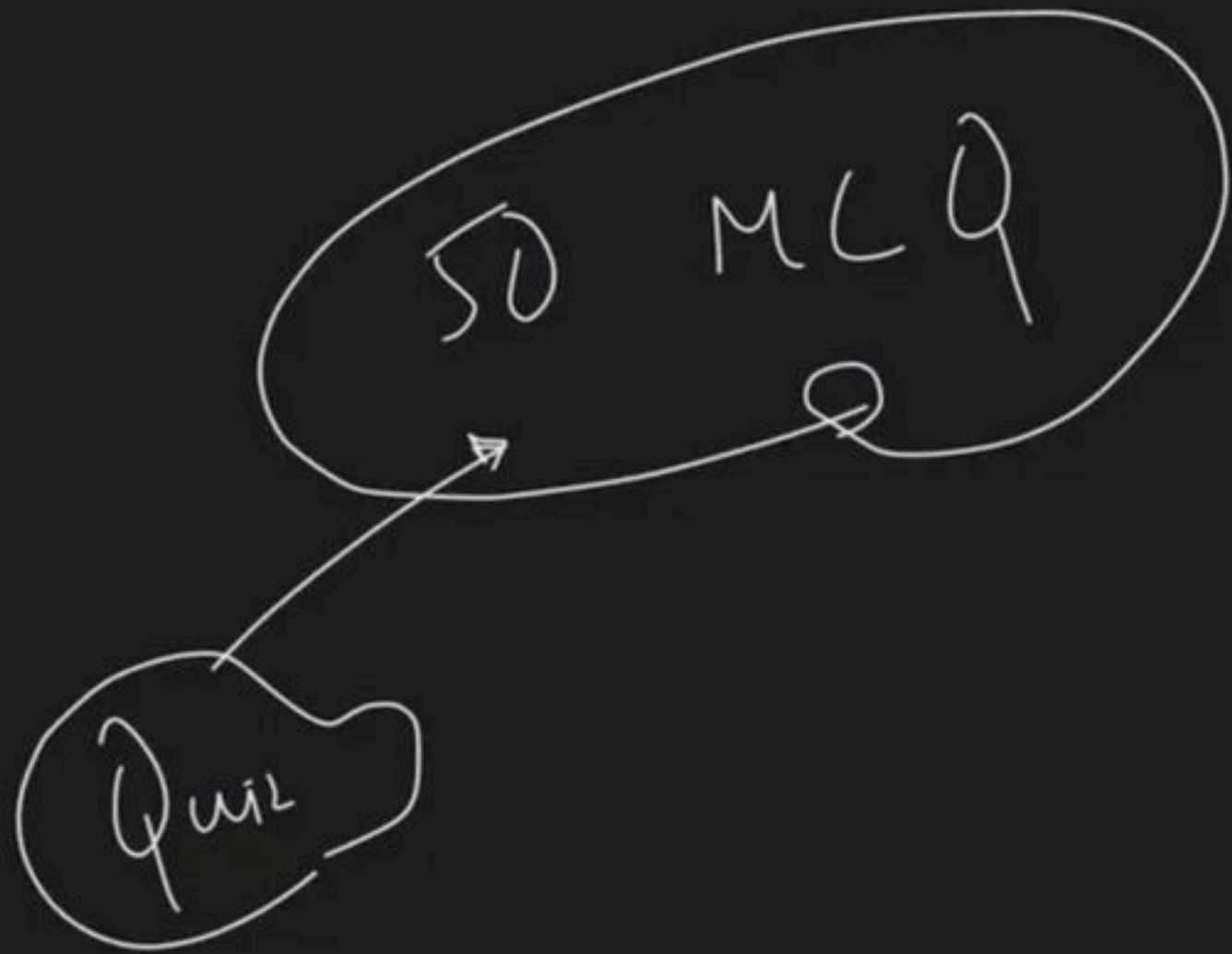
~~int *p~~

int *p

copy
banti
hai.



3



① → Anif → ~~1st~~ week khata

↳

Next week
=

int(*fp) (char* ...)

```

=> int len(char* str) {
    return strlen(str);
}

```

```
main ( )
{
}
}
```

```
char str[] = "code help";
```

$$\text{count} \leq \ln(\text{str}) \leq$$

Handwritten diagram illustrating the components of a function signature: `int(*fp)(char*)`.

- Return Type:** `int` is identified as the return type.
- Argument:** `char*` is identified as the argument (arg).
- Function Pointer Name:** `(*fp)` is identified as the name of the function pointer (FP).

The diagram also includes checkmarks above the signature and the words "Type" and "Name of FP" written below the signature.

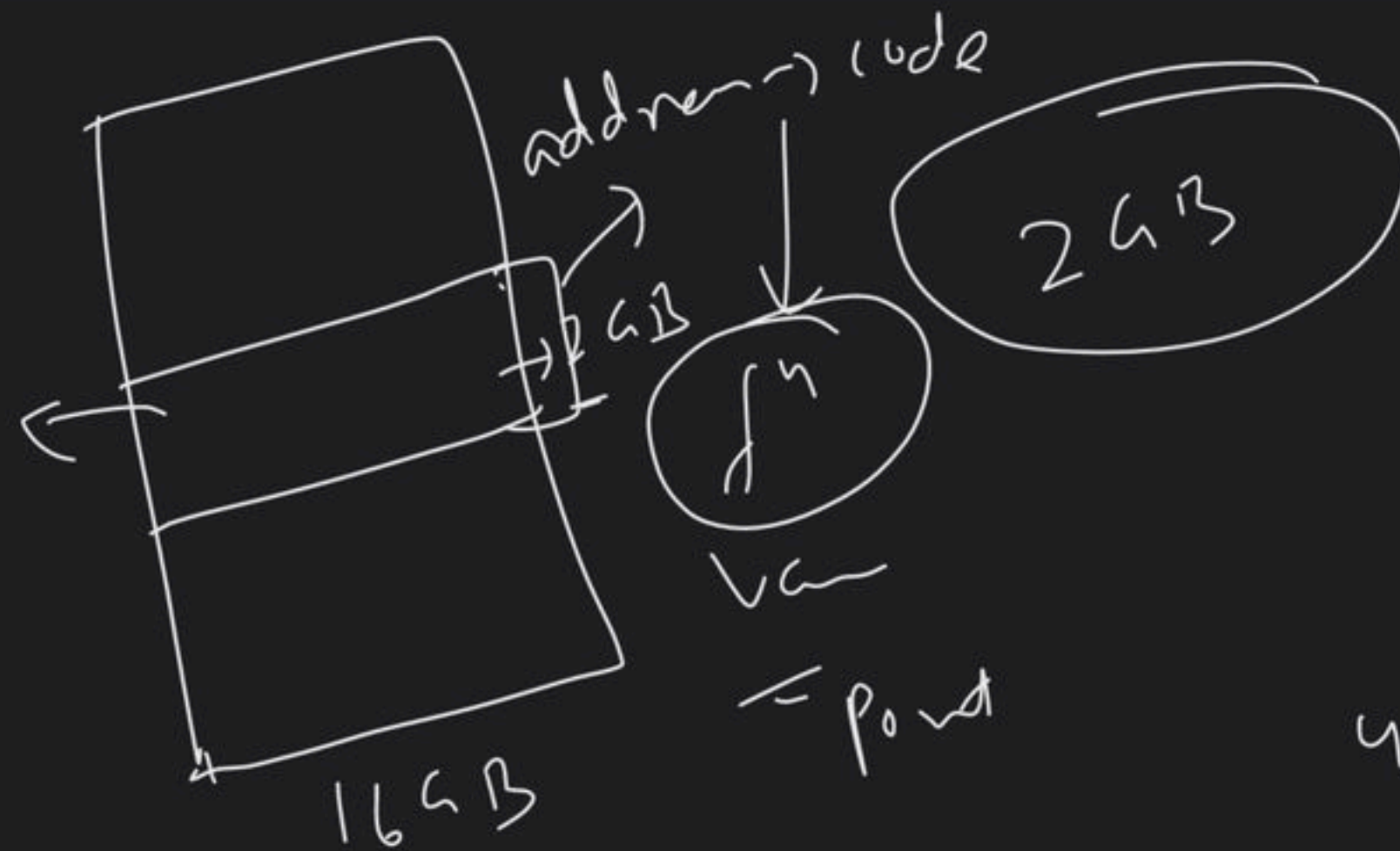
$f_p = \text{len};$
 $f_p = 4\text{len};$

fp(str) ==

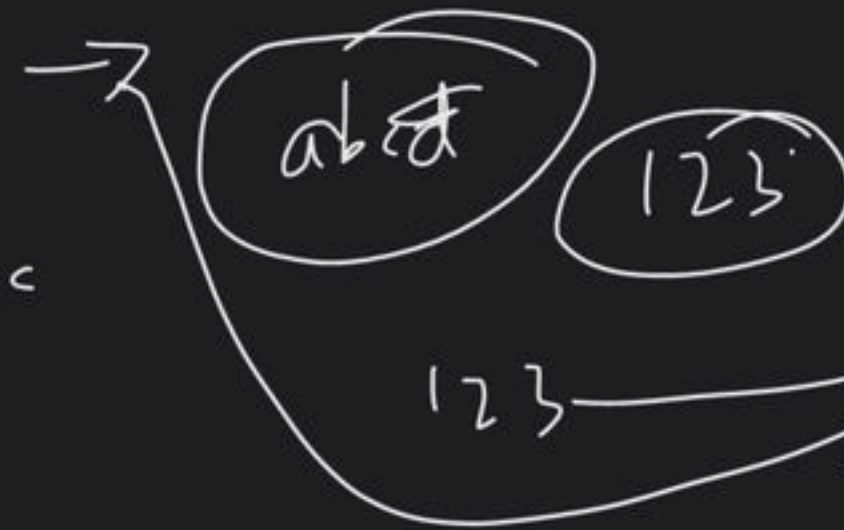
int(*fp)(char*)

len(str) <

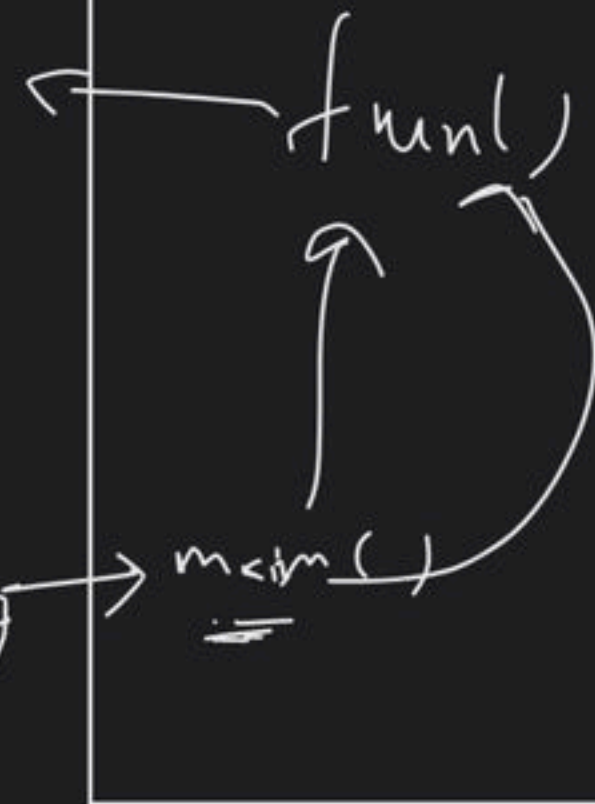
fp(str)



OS



405



Min →

→ Recursion →

Opp → X

↓

500

1000

will x
skill

1 year →

↓

2 Quater →

1

